ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA



LG DUT CYBER SECURITY LAB

BÁO CÁO THỰC TẬP TUẦN 1

Thành viên Nhóm 4

1. Nguyễn Trường Bảo
2. Mai Thị Mỹ Đình
3. Bùi Phương Chi
4. Nguyễn Đình Nhất
5. Đỗ Hữu Dũng
6. Nguyễn Thành Lưu

Đà Nẵng, ngày . . . tháng . . . năm . . .

Mục lục

1 Cơ sở lý thuyết và phân tích kỹ thuật **1**

1.1 Tổng quan về webOS Open Source Edition . . . 1
1.2 Yocto Project, OpenEmbedded và BitBake . . . 1
1.3 Phân tích tài nguyên khi build webOS . . . 2
1.4 SSH, MobaXterm và xác định địa chỉ bằng ip route . . . 3
1.5 SMACK và Mandatory Access Control . . . 3
1.6 Netfilter, iptables và extension trong kernel . . . 4
1.7 Bảng, chain và đường đi của packet . . . 5
1.8 Stateful firewall và connection tracking . . . 6
1.9 Cơ sở lý thuyết các cuộc tấn công được kiểm thử . . . 6
1.10 Mô hình triển khai thực tế của nhóm . . . 7
1.11 Phương pháp đánh giá hiệu quả firewall . . . 7
1.12 Kết luận của phần cơ sở lý thuyết . . . 7

2 Hướng dẫn tích hợp SMACK lên webOS cho Raspberry Pi 4 **8**

2.1 Giới thiệu . . . 8
2.2 Thực hiện . . . 8
2.3 Cập nhật và nâng cấp hệ thống . . . 8
2.4 .Cài đặt các gói phụ thuộc . . . 8
2.5 Tải mã nguồn webOS OSE . . . 9
2.6 Cài đặt và cấu hình môi trường xây dựng . . . 9
2.7 Chỉnh sửa cấu hình . . . 9
2.8 Cấu hình kernel . . . 9
2.9 Cập nhật image . . . 10
2.10 Cấu hình phân phối . . . 11
2.11 Cập nhật các URL nguồn . . . 11
2.12 Vá lỗi SMACK . . . 13
2.13 Xây dựng Image . . . 15
2.14 Kiểm tra kết quả . . . 15
2.15 Các lỗi trong quá trình Build WebOS . . . 17
2.16 Lỗi do thiết lập cho việc build quá nhiều nhân, nhiều luồng . . . 17
2.17 Lỗi qt_base do cấu hình meta-qt6 và tràn RAM . . . 18
2.18 Lỗi phiên bản của aktualizr . . . 19
2.19 Quy trình build lại an toàn . . . 20

3 Phát triển tường lửa **20**

3.1 Yêu cầu . . . 20

3.2 Các rule iptables . . . 21

3.3 Thử nghiệm . . . 22

3.3.1 SYN Flood Attack . . . 22

3.3.2 LAND Attack . . . 24

3.3.3 Smurf Attack . . . 25

3.3.4 Random Source Attack . . . 28

3.4 Triển khai bash shell iptables . . . 30

3.5 Các tham số cấu hình iptables . . . 32

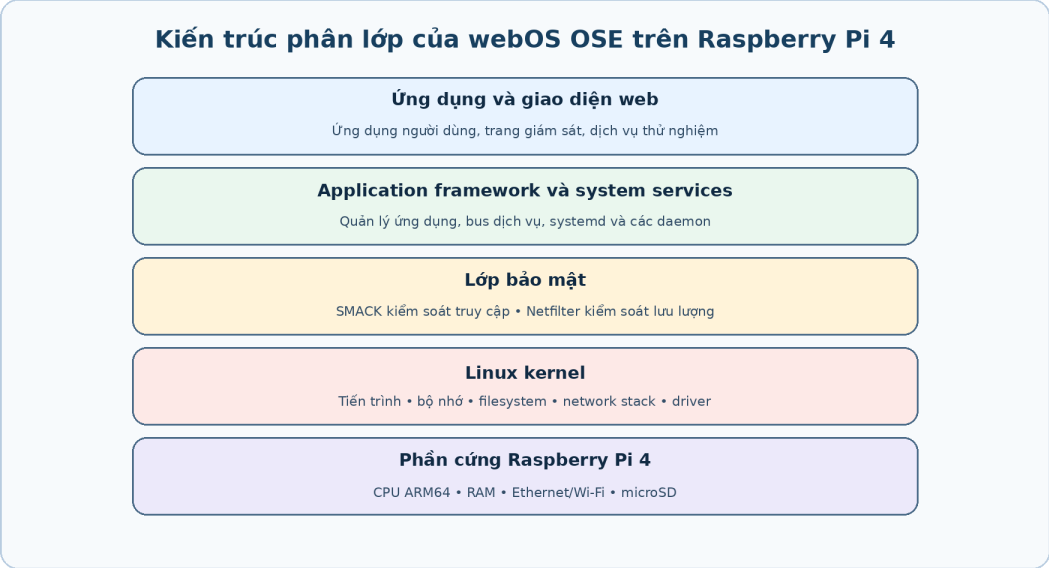
3.6 So sánh khả năng hỗ trợ iptables . . . 33

1 Cơ sở lý thuyết và phân tích kỹ thuật

1.1 Tổng quan về webOS Open Source Edition

WebOS Open Source Edition (webOS OSE) là hệ điều hành mã nguồn mở do LG phát triển trên nền Linux, hướng đến các thiết bị thông minh và hệ thống nhúng. Khác với Ubuntu Desktop, webOS không được hoàn thiện bằng cách cài từng ứng dụng trực tiếp trên Raspberry Pi. Kernel, thư viện, dịch vụ hệ thống, chính sách bảo mật và ứng dụng cần thiết được lựa chọn ngay trong hệ thống build, sau đó đóng gói thành một image thống nhất để ghi vào thẻ nhớ.

Cách xây dựng này giúp kiểm soát chính xác thành phần tồn tại trên thiết bị, giảm package không cần thiết và tạo môi trường có thể tái lập. Đổi lại, nếu cần thêm một chức năng ở mức kernel như SMACK, connbytes, hashlimit hoặc nftables thì phải cấu hình và build lại image; chỉ sao chép một chương trình userspace vào thiết bị thường chưa đủ.



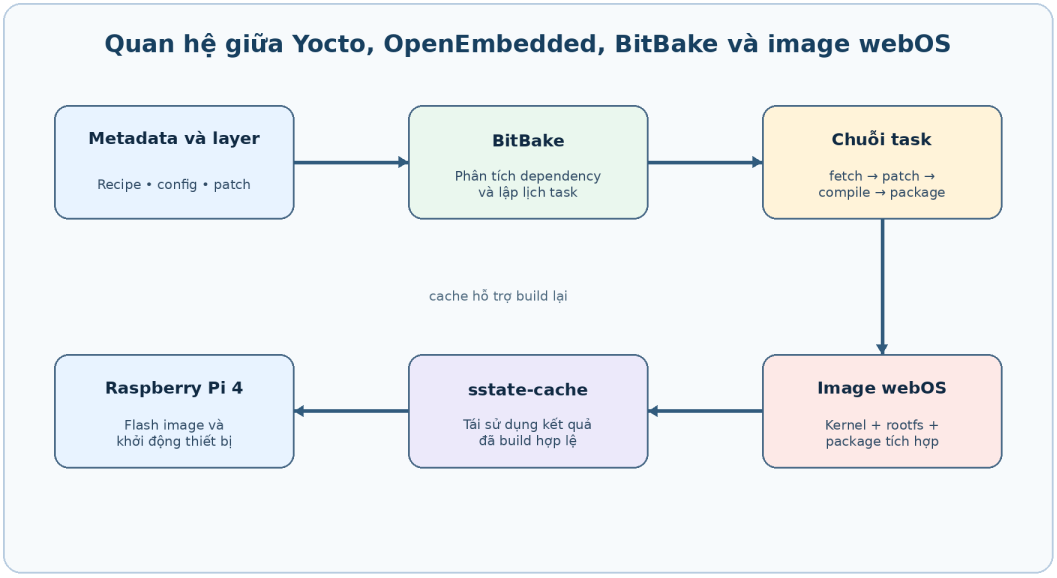
Hình 1: Kiến trúc phân lớp của webOS OSE trên Raspberry Pi 4.

Mỗi lớp có một nhiệm vụ riêng nhưng liên hệ chặt chẽ. Ứng dụng gửi yêu cầu đến framework và dịch vụ hệ thống; dịch vụ sử dụng system call để yêu cầu kernel quản lý file, bộ nhớ và mạng. SMACK kiểm soát quyền truy cập giữa chủ thể và đối tượng, còn Netfilter kiểm soát packet đi qua network stack. Vì vậy SMACK và firewall không thay thế nhau: một cơ chế bảo vệ tài nguyên nội bộ, cơ chế còn lại bảo vệ bề mặt tiếp xúc qua mạng.

1.2 Yocto Project, OpenEmbedded và BitBake

Yocto Project cung cấp công cụ và quy trình để tạo một bản Linux tùy chỉnh cho thiết bị nhúng. OpenEmbedded cung cấp metadata dưới dạng recipe, class, configuration và layer. BitBake đọc metadata, giải quyết dependency và thực thi các task theo đúng thứ tự. Ba thành phần này tạo thành hệ thống build của webOS OSE.

Thành phần	Chức năng	Liên hệ với dự án
Recipe (.bb)	Mô tả nguồn tải, phiên bản, dependency, patch và các bước build.	Dùng để build kernel, systemd, iptables và package bổ sung.
Layer	Tổ chức recipe/config theo nền tảng hoặc chức năng.	Kết hợp layer webOS, Raspberry Pi, SMACK và OpenEmbedded.
BitBake task	Từng công đoạn fetch, patch, configure, compile, install, package.	Log task là nơi xác định nguyên nhân gốc khi build lỗi.
sstate-cache	Lưu kết quả trung gian có thể tái sử dụng.	Giảm thời gian build lại; không nên xóa toàn bộ khi chưa cần.
Image recipe	Quyết định package trong root filesystem.	Bổ sung htop, tcpdump và extension firewall vào image.



Hình 2: Quan hệ giữa metadata, BitBake, cache và image webOS.

Chuỗi task thông thường gồm `do_fetch`, `do_unpack`, `do_patch`, `do_configure`, `do_compile`, `do_install`, `do_package` và `do_rootfs`. Một lỗi ở `do_compile` khác hoàn toàn lỗi `do_fetch`. Do đó không nên chỉ nhìn dòng FAILED cuối terminal; cần mở log task bị lỗi và tìm thông báo đầu tiên như "fatal error", "Killed", "No space left", "Nothing PROVIDES" hoặc checksum mismatch.

1.3 Phân tích tài nguyên khi build webOS

Build webOS tạo nhiều tiến trình compiler C/C++ chạy đồng thời. `BB_NUMBER_THREADS` kiểm soát số task BitBake đồng thời, còn `PARALLEL_MAKE` hoặc tham số `-j` kiểm soát số job bên trong một task. Nếu cả hai đều quá cao, số compiler có thể tăng mạnh, làm CPU đạt 100%, RAM cạn và ổ đĩa chịu tải I/O lớn.

Công cụ	Theo dõi	Ý nghĩa chẩn đoán
htop	CPU, RAM, load average, process	Xác định compiler hoặc BitBake task sử dụng nhiều tài nguyên.
free -h	available RAM và swap	Đánh giá bộ nhớ thực còn có thể cấp phát; không chỉ nhìn cột free.
df -h	Dung lượng filesystem	Phát hiện BUILD, downloads hoặc sstate-cache làm đầy SSD.
df -i	Số inode	Yocto tạo nhiều file nhỏ; có thể hết inode trước khi hết GB.
dmesg	OOM, I/O, kernel warning	Xác nhận OOM killer đã kết thúc cc1plus hoặc có lỗi filesystem.

Thông báo "Killed signal terminated program cc1plus" thường cho biết compiler bị kernel kết thúc do áp lực bộ nhớ, không phải khẳng định mã nguồn Qt bị sai. Cách xử lý đúng là kiểm tra dmesg, RAM/swap, giảm mức song song và chỉ build lại recipe cần thiết. Xóa toàn bộ BUILD hoặc cache làm mất dữ liệu tái sử dụng và có thể kéo dài quá trình mà không xử lý nguyên nhân gốc.

1.4 SSH, MobaXterm và xác định địa chỉ bằng ip route

SSH cung cấp kênh quản trị có mã hóa từ máy cá nhân đến Ubuntu Server hoặc Raspberry Pi. MobaXterm là ứng dụng phía client giúp quản lý phiên SSH, SFTP và nhiều terminal; nó không thay đổi giao thức SSH. Lệnh ip route cho biết tuyến mặc định, gateway, interface và địa chỉ nguồn mà Linux ưu tiên sử dụng.

Trường trong ip route	Ý nghĩa	Ví dụ
default	Tuyến dùng khi không có tuyến cụ thể hơn.	default via 10.10.16.1
via	Gateway hoặc next hop.	10.10.16.1
dev	Interface được dùng để gửi packet.	eno1
src	IP nguồn ưu tiên của máy.	10.10.16.245
/24	Độ dài prefix, tương ứng mask 255.255.255.0.	10.10.16.0/24

IP dùng để SSH phải có đường đi hai chiều từ máy người dùng đến máy đích. Mật khẩu dùng chung của Lab không nên xuất hiện trong bản báo cáo phát hành, ảnh chụp terminal hoặc kho mã nguồn.

1.5 SMACK và Mandatory Access Control

Linux truyền thống sử dụng Discretionary Access Control, trong đó quyền truy cập chủ yếu dựa trên user, group và mode bit. Mandatory Access Control bổ sung chính sách bắt buộc mà ứng dụng không thể tự ý bỏ qua. SMACK (Simplified Mandatory Access Control Kernel) là một Linux Security Module sử dụng nhãn để kiểm soát tương tác giữa process, file, socket và dịch vụ.

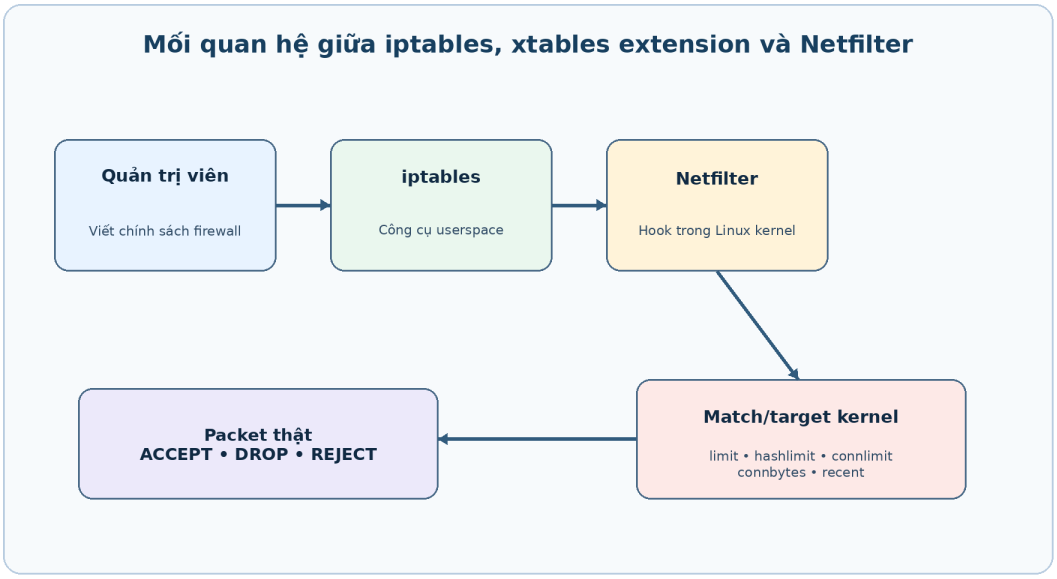
Khi một ứng dụng web bị khai thác, quyền của tiến trình không chỉ phụ thuộc tài khoản đang chạy mà còn bị giới hạn bởi nhãn SMACK và rule giữa các nhãn. SMACK vì vậy giúp giảm khả năng kẻ tấn công di chuyển từ dịch vụ bị xâm nhập sang file hoặc service khác.

CONFIG_SECURITY_SMACK bật cơ chế trong kernel; CONFIG_TMPFS_XATTR cho phép lưu nhãn trên tmpfs; cấu hình LSM quyết định SMACK có thực sự được kích hoạt trong chuỗi module bảo mật hay không.

Cơ chế	Đối tượng bảo vệ	Ví dụ trong dự án
SMACK	Process, file, socket, service	Ngăn ứng dụng đọc/ghi tài nguyên không đúng nhãn.
Netfilter/firewall	Packet và kết nối mạng	Chặn SYN bất thường, giới hạn request đến web server.
Quyền Unix	User, group, mode bit	Hạn chế file theo tài khoản hệ thống.
Cấu hình dịch vụ	Chức năng của web/SSH	Giới hạn tài khoản, port và quyền của daemon.

1.6 Netfilter, iptables và extension trong kernel

Netfilter là framework xử lý packet nằm trong Linux kernel. iptables là công cụ userspace dùng để mô tả rule và gửi cấu hình xuống kernel; iptables không trực tiếp lọc packet. Các match như limit, hashlimit, connlimit, connbytes và recent cần extension tương ứng trong kernel, đồng thời userspace phải có thư viện xtables hiểu cú pháp đó.



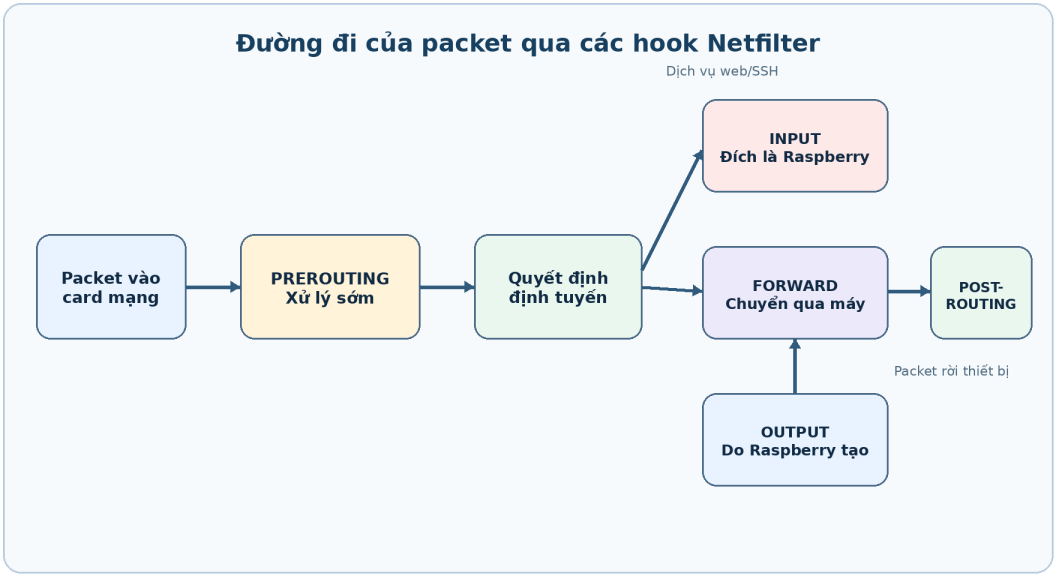
Hình 3: Quan hệ giữa người quản trị, iptables và Netfilter trong kernel.

Cấu hình =y nghĩa là extension được tích hợp thẳng vào kernel; =m nghĩa là tạo module cần được đóng gói và nạp. Nếu kernel thiếu extension, rule có thể báo "No chain/target/match by that name". Nếu chỉ có kernel module nhưng image thiếu thành phần userspace, iptables vẫn có thể không hiểu tham số. Vì vậy phải xác minh cả kernel config, module/package trong image và counter của rule chạy thực tế.

Extension	Chức năng	Giới hạn cần lưu ý
limit	Token bucket chung cho toàn rule.	Không mặc nhiên giới hạn riêng từng IP; attacker có thể tiêu hết quota chung.
hashlimit	Giới hạn theo srcip/dstip/port.	srcip kém hiệu quả trước nhiều địa chỉ giả mạo ngẫu nhiên.
connlimit	Giới hạn số kết nối đồng thời.	Ngưỡng quá thấp có thể chặn NAT có nhiều người dùng chung IP.
connbytes	So khớp byte/packet của một kết nối.	Phụ thuộc connection tracking và không phải rate limiter.
recent	Ghi nhớ IP xuất hiện gần đây.	Cần chọn hitcount/seconds hợp lý để tránh false positive.

1.7 Bảng, chain và đường đi của packet

Bảng filter thực hiện chính sách cho phép/chặn; nat thay đổi địa chỉ hoặc cổng; mangle dùng cho đánh dấu hoặc điều chỉnh packet; raw can thiệp trước connection tracking. Đối với Raspberry là máy đích chạy web và SSH, phần lớn chính sách nên tập trung tại filter/INPUT. Rule trong mangle/PREROUTING chỉ nên dùng khi thực sự cần xử lý sớm vì packet bị DROP tại đó sẽ không bao giờ tới INPUT.



Hình 4: Đường đi của packet qua các hook Netfilter.

Luồng packet	Hook chính	Trường hợp trong dự án
Đến Raspberry	PREROUTING → INPUT	Kali truy cập HTTP/SSH hoặc gửi ICMP đến Raspberry.
Qua Raspberry	PREROUTING → FORWARD → POSTROUTING	Chỉ xảy ra khi Raspberry hoạt động như router/gateway.
Từ Raspberry	OUTPUT → POSTROUTING	WebOS gửi DNS, HTTP hoặc phản hồi SYN-ACK.

Rule trong một chain được xét từ trên xuống. Khi packet khớp ACCEPT, DROP hoặc REJECT, việc xét thường kết thúc. Do đó rule chống LAND đặt sau rule ACCEPT port 80 không thể chặn

các packet đã được chấp nhận. Thứ tự rule phải phản ánh mức ưu tiên bảo vệ, không chỉ đơn thuần là tập hợp các lệnh đúng cú pháp.

1.8 Stateful firewall và connection tracking

Connection tracking giúp kernel phân loại packet thành NEW, ESTABLISHED, RELATED hoặc INVALID. ESTABLISHED là packet thuộc kết nối đã được nhận biết; RELATED là luồng mới liên quan đến kết nối hiện có; INVALID là packet không thể gán hợp lệ vào trạng thái nào. Cho phép ESTABLISHED,RELATED giúp phản hồi của SSH, HTTP và các kết nối do Raspberry chủ động tạo quay lại bình thường.

Conntrack cũng tiêu thụ RAM. Trong SYN flood hoặc random-source flood, số entry có thể tăng và trở thành mục tiêu cạn kiệt tài nguyên. Vì vậy đánh giá firewall cần theo dõi SYN_RECV, số entry conntrack, packet rate và log kernel, không chỉ nhìn CPU tổng.

1.9 Cơ sở lý thuyết các cuộc tấn công được kiểm thử

- SYN Flood

TCP thiết lập kết nối qua ba bước SYN → SYN-ACK → ACK. SYN flood gửi nhiều SYN nhưng không hoàn thành ACK cuối, khiến server duy trì trạng thái chờ trong SYN backlog và có thể tạo entry conntrack. Linux hiện đại sử dụng SYN cookies để giảm nguy cơ cạn backlog, nhưng vẫn phải nhận và xử lý packet. Hiệu quả phòng vệ phải được đánh giá qua khả năng phục vụ HTTP, số trạng thái SYN_RECV, CPU ksoftirqd, packet rate và counter firewall.

- Random-source SYN Flood

Tùy chọn –rand-source tạo nhiều IP nguồn giả mạo nhằm làm giảm hiệu quả của cơ chế chỉ giới hạn theo srcip. Một máy Kali tạo traffic với nhiều source IP vẫn là DoS có spoofing hoặc mô phỏng đặc điểm nhiều nguồn, chưa phải DDoS phân tán thật. Rule chống spoofing phải gắn với interface và ranh giới mạng; chặn toàn bộ 192.168.0.0/16 trên interface LAN sẽ chặn chính các máy quản trị hợp lệ.

- Smurf Attack và ICMP Flood

Smurf cổ điển gửi ICMP Echo Request đến broadcast và giả mạo source thành IP nạn nhân. Nhiều host trong mạng khuếch đại phải cùng trả lời thì amplification mới xuất hiện. Hệ điều hành hiện đại thường bỏ qua ICMP broadcast và router không forward directed broadcast, nên thử nghiệm của nhóm có thể chỉ chứng minh được bước phát packet giả mạo. Không nên chặn toàn bộ ICMP vì ICMP còn phục vụ báo lỗi mạng và Path MTU Discovery; nên giới hạn riêng Echo Request.

- LAND Attack

LAND cổ điển yêu cầu source IP bằng destination IP và source port bằng destination port. Chỉ sử dụng -a để giả mạo source IP chưa tái tạo đầy đủ điều kiện này. Linux hiện đại thường không còn lỗ hổng treo máy cổ điển, vì vậy bài lab nên tập trung vào khả năng phát hiện packet bất thường và counter của rule. Nếu rule LAND đặt sau ACCEPT cho port 22 hoặc 80, packet có thể được cho qua trước khi tới rule chặn.

- Port scanning

Port scanning gửi probe để phân biệt port open, closed hoặc filtered. SYN, connect, FIN, NULL và XMAS scan sử dụng các mẫu packet khác nhau. Rule TCP flag chỉ xử lý một số scan bất thường, không chặn mọi kỹ thuật. Custom chain port-scanning chỉ hoạt động khi INPUT có rule nhảy vào chain đó; tạo chain nhưng không gọi sẽ không nhận packet.

1.10 Mô hình triển khai thực tế của nhóm

Ubuntu Server lg_lab_server7 tại 10.10.16.245 được sử dụng để tải source và build image. Raspberry Pi 4 là máy đích chạy webOS, dịch vụ web, SSH và firewall. Kali hoặc laptop đóng vai trò máy quản trị và máy tạo lưu lượng kiểm thử. Hai địa chỉ 192.168.1.185 và 10.10.47.44 của Raspberry phải được gắn với interface/subnet cụ thể trước khi viết rule; không được giả định cả hai luôn hoạt động trên cùng một đường mạng.

Mô hình này là host-based firewall: rule bảo vệ chính Raspberry, không đại diện cho một firewall biên của doanh nghiệp. Do đó các rule anti-spoofing được sao chép từ server Internet có thể không phù hợp. Ví dụ, chặn nguồn 192.168.0.0/16 tại PREROUTING sẽ làm mất SSH và HTTP từ chính mạng LAN 192.168.1.0/24.

1.11 Phương pháp đánh giá hiệu quả firewall

Không thể kết luận Raspberry "sập" chỉ vì không ping được, vì ICMP có thể bị firewall chặn. Mất SSH cũng có thể do rule sai, packet loss hoặc interface nghẽn. Một thử nghiệm có giá trị phải giữ nguyên máy phát, loại packet, port, kích thước, thời lượng và dịch vụ; chỉ thay đổi trạng thái firewall.

Nhóm bằng chứng	Trước tấn công	Trong tấn công	Sau khi bật firewall
Dịch vụ	HTTP status/latency; mở SSH được.	Timeout, tỷ lệ lỗi, khả năng mở phiên mới.	So sánh cùng tải và thời lượng.
Tài nguyên	CPU, RAM, load nền.	CPU process/ksoftirqd, RAM, conntrack.	Mức giảm và khả năng phục hồi.
Mạng	Packet/s, TCP state nền.	SYN_RECV, packet/s, retransmission.	Packet nhận và packet bị drop.
Firewall	Lưu ruleset, reset counter.	Counter của đúng rule tăng.	Xác định rule tạo khác biệt.

Bộ đếm phải được reset trước từng ca kiểm thử. Nếu counter của rule không tăng, không được khẳng định rule đó đã chặn tấn công. Đồng thời cần có rollback trước khi nạp rule từ xa, vì một lỗi thứ tự hoặc anti-spoofing sai có thể tự khóa phiên SSH.

1.12 Kết luận của phần cơ sở lý thuyết

Quá trình xây dựng webOS và triển khai firewall là mối liên hệ giữa hệ thống build, kernel, cơ chế kiểm soát truy cập và network stack. Yocto/OpenEmbedded/BitBake tạo image; SMACK bảo vệ tài nguyên nội bộ; Netfilter xử lý packet; iptables cung cấp giao diện cấu hình; extension kernel quyết định các match nâng cao có sử dụng được hay không. Vì vậy một rule đúng cú pháp chưa đủ: rule phải phù hợp topology, đúng hook, đúng thứ tự, có ngưỡng được kiểm chứng và không gây false positive đối với lưu lượng hợp lệ.

2 Hướng dẫn tích hợp SMACK lên webOS cho Raspberry Pi 4

2.1 Giới thiệu

- WebOS Open Source Edition (OSE) là hệ điều hành mã nguồn mở của LG, kế thừa kiến trúc từ Palm webOS, chạy trên Linux và tối ưu cho thiết bị nhúng như Raspberry Pi.
- SMACK là cơ chế Mandatory Access Control (MAC) trong Linux, tương tự `SELinux/AppArmor` nhưng đơn giản hơn, do Samsung phát triển và webOS OSE tích hợp mặc định.

2.2 Thực hiện

- Trước khi bắt đầu quá trình xây dựng, cần thiết lập môi trường làm việc phù hợp trên hệ thống Linux (khuyến nghị sử dụng Ubuntu server).
- Sử dụng Ubuntu Server LTS 22.04 hoặc 24.04, cập nhật hệ thống.
- Cài đặt đầy đủ gói phụ thuộc (dependencies) cho Yocto/BitBake (build-essential, gcc/g++, make, cmake, git, wget, gawk, chrpath, curl, tar và các thư viện liên quan).
- Đảm bảo dung lượng trống, ưu tiên filesystem ext4.
- RAM hệ thống tối thiểu 16GB; nếu thấp hơn cần cấu hình Swap 8–16GB để tránh lỗi OOM.
- Tạo user không phải root để thực hiện build, có quyền ghi trong workspace.
- Kiểm tra kết nối mạng ổn định để tải source (do_fetch).
- Cấu hình build đa luồng hợp lý theo số nhân CPU

(BB_NUMBER_THREADS, -j cho make).

- Cài đặt SSH client MobaXterm để thao tác từ xa.
- Chuẩn bị workspace lưu trữ source code và download cache (DL_DIR) để tái sử dụng.
- Thông tin truy cập máy chủ

Thông tin	Giá trị	Ghi chú
Tên tài khoản	lg_lab_serverX	X từ 1 đến 10
Máy sử dụng trong báo cáo	lg_lab_server7	Ubuntu Server tại Lab
IP máy lg_lab_server7	10.10.16.245	Dùng để SSH/MobaXterm
Mật khẩu Lab	lgedv2024	Dùng chung cho hầu hết thiết bị tại Lab

2.3 Cập nhật và nâng cấp hệ thống

2.4 .Cài đặt các gói phụ thuộc

- Cài đặt các gói cần thiết cho quá trình xây dựng:

Lệnh	Ý nghĩa
sudo apt install build-essential snake git curl	Cài các công cụ cần thiết để build phần mềm, làm việc với Git, và tải dữ liệu trong terminal.
sudo apt install autoconf automake bison flex gawk libtool pkg-config	Cài các công cụ hỗ trợ tạo hệ thống build, phân tích cú pháp và quản lý thư viện, cần cho quá trình build phần mềm/OS trên Linux.

2.5 Tải mã nguồn webOS OSE

- Tải mã nguồn webOS OSE phiên bản 2.20 từ kho lưu trữ GitHub:

Lệnh	Ý nghĩa
sudo apt install git-all	Cài phiên bản Git hoàn chỉnh với tất cả công cụ hỗ trợ đi kèm
git clone https://github.com/webososose/build-webos.git	Tải toàn bộ mã nguồn và cấu trúc của dự án build-webos từ GitHub về máy để bắt đầu quá trình build hệ điều hành.
cd build-webos	Di chuyển terminal vào thư mục build-webos để tiếp tục thao tác với mã nguồn hoặc chạy lệnh build.
git checkout 2.20	Chuyển repo sang phiên bản/nhánh 2.20 để build hoặc phát triển đúng theo code của phiên bản đó.

2.6 Cài đặt và cấu hình môi trường xây dựng

- Chạy các lệnh để cài đặt các công cụ cần thiết và cấu hình môi trường xây dựng cho Raspberry Pi 4:

Lệnh	Ý nghĩa
sudo scripts/prerequisites.sh	Chạy script để tự động thiết lập các điều kiện cần thiết trước khi build dự án.
./mcf-p 4-b 3 raspberrypi4-64	Chạy script build WebOS với 6 luồng song song và thiết lập máy đích là Raspberry Pi 4 64-bit.

2.7 Chỉnh sửa cấu hình

Để tích hợp SMACK và sửa các lỗi liên quan đến lệnh fetch trong các tệp bitbake, cần thực hiện các thay đổi sau:

2.8 Cấu hình kernel

- Trong thư mục:

```
/build-webos/meta-webososose/meta-webos-raspberrypi/recipes-kernel/linux/linux-rasp⌋
↪ berrypi
```

Thêm vào tệp bridge.cfg:

```
CONFIG_NETFILTER_XT_MATCH_CONNBYTES=y
CONFIG_NETFILTER_XT_MATCH_CONNLIMIT=y
CONFIG_NETFILTER_XT_MATCH_IPRANGE=y
CONFIG_NETFILTER_XT_MATCH_LENGTH=y
CONFIG_NETFILTER_XT_MATCH_LIMIT=y
CONFIG_NETFILTER_XT_MATCH_MAC=y
```

```
CONFIG_NETFILTER_XT_MATCH_QUOTA=y
CONFIG_NETFILTER_XT_TARGET_NFLOG=y
CONFIG_NF_LOG_IPV4=y
CONFIG_NETFILTER_XT_MATCH_RECENT=y
CONFIG_NETFILTER_XT_MATCH_TCPMSS=y
CONFIG_NETFILTER_XT_TARGET_TCPMSS=y
CONFIG_NETFILTER_XT_MATCH_HASHLIMIT=y
CONFIG_NETFILTER_XT_MATCH_NFACCT=y
```

Thêm vào tập tin security.cfg

```
CONFIG_SECURITY_SMACK=y
CONFIG_DEFAULT_SECURITY="smack"
CONFIG_DEFAULT_SECURITY_SMACK=y
CONFIG_TMPFS_XATTR=y
CONFIG_SECURITY_SMACK_BRINGUP=y
CONFIG_LSM="lockdown,yama,loadpin,safesetid,integrity,smack,selinux,tomoyo,apparm
↪  or"
# kernel config to enable access to /proc/config.gz
CONFIG_IKCONFIG_PROC=y
CONFIG_IKCONFIG=y
CONFIG_PROC_FS=y
CONFIG_EXPERT=y
```

- Các thay đổi kernel config là cần thiết vì:
- Kernel mặc định trong layer Raspberry Pi của WebOS chưa bật stack bảo mật SMACK, nên không đáp ứng yêu cầu bảo mật MAC của WebOS OSE.
- WebOS cần SMACK để kiểm soát quyền truy cập giữa ứng dụng và service hệ thống (Mandatory Access Control).
- Kernel cần hỗ trợ xattr trên tmpfs để gán nhãn bảo mật SMACK ngay khi khởi động (bring-up), tránh lỗi permission ở môi trường embedded.
- Các module NETFILTER_XT_* phải bật để kernel hỗ trợ rule firewall nâng cao bằng iptables (giới hạn kết nối, lọc theo MAC/IP range, ghi log gói tin).
- Việc bật Netfilter giúp tránh lỗi mạng và nghẽn kết nối khi chạy ứng dụng

nhúng trên Raspberry Pi.

- Kích hoạt /proc/config.gz giúp kiểm tra lại config kernel sau khi build, phục vụ debug và hỗ trợ sửa lỗi do_fetch, pipeline BitBake ổn định hơn.

2.9 Cập nhật image

- Trong thư mục /build-webos/meta-webososе/meta-webos/recipes-core/images, thêm vào tệp webos-image.bb:

```
WEBOS_IMAGE_EXTRA_INSTALL:append = " \
    htop \
    tcpdump \
    kernel-module-xt-hashlimit \
    ipset \
```

```
    apt \
"
```

Mục đích:

- Bổ sung các công cụ giám sát hệ thống (htop) để kiểm tra tài nguyên khi build và khi thiết bị chạy.
- Thêm công cụ phân tích mạng (tcpdump, tcpdump) để bắt và debug gói tin trực tiếp trên thiết bị nhúng.
- Cài module kernel Netfilter (xt-hashlimit) để hỗ trợ rule giới hạn kết nối trong firewall (iptables cần module này).
- Thêm ipset/ipset giúp quản lý tập IP/MAC hiệu quả, tăng tốc xử lý rule

firewall trên embedded.

- Bổ sung apt nhằm hỗ trợ cài gói bổ sung nhanh khi cần test/debug mà không phải build lại image.
- Các gói này giúp quá trình bring-up, debug và vận hành WebOS trên Raspberry Pi ổn định và thuận tiện hơn.

2.10 Cấu hình phân phối

- Trong thư mục /build-webos/conf, thêm vào tệp local.conf:

```
DISTRO_FEATURES:append = " smack"
DISTRO_FEATURES:append = " smack-bringup"
```

- Phải thêm vào DISTRO_FEATURES vì:
- Bật SMACK ở cấp hệ thống (distro) để toàn pipeline Yocto/BitBake kích hoạt cơ chế bảo mật MAC mà WebOS OSE yêu cầu.
- Kích hoạt smack-bringup giúp hỗ trợ giai đoạn khởi động ban đầu (OS bring-up) khi SMACK chưa có đầy đủ label/rule, tránh lỗi permission làm build/boot fail.
- Đảm bảo các layer và recipe của WebOS được build kèm hỗ trợ SMACK đúng cách, không phải chỉnh thủ công sau khi tạo image.

2.11 Cập nhật các URL nguồn

- Trong/build-webos/meta-webososе/meta-webos/recipes-webos/localization-

tool, thay thế SRC_URI trong localization-tool-native.bb:

```
SRC_URI="git://github.com/iLib-js/ilib-loctool-webos-dist.git;branch=main;protocol=https"
```

Mục đích:

- Đường dẫn SRC_URI mặc định có thể đã lỗi hoặc không còn hoạt động, gây fail ở bước do_fetch.
- Cần thay bằng link mới để tải đúng mã nguồn ilib-loctool đang được duy trì.
- Chọn branch=main để lấy phiên bản mới nhất, tương thích WebOSE.
- Dùng protocol=https giúp kết nối ổn định và an toàn hơn khi clone repo từ GitHub.
- Mục đích chung: đảm bảo BitBake fetch thành công, không gián đoạn pipeline build.
- Trong `/build-webos/meta-webososse/meta-webos/recipes-upstreamable/arm-compute-library`, thay thế SRCBRANCH trong `arm-compute-library_22.08.bb`:

```
SRCBRANCH = "main"
```

Mục đích:

- Nhánh mặc định của recipe có thể đã cũ hoặc không tồn tại, làm BitBake không fetch được source.
- Đổi SRCBRANCH = "main" để lấy đúng nhánh đang được duy trì trên

upstream.

- Giúp đảm bảo tính tương thích với pipeline build hiện tại của WebOS

OSE.

- Mục đích chung: tránh lỗi do_fetch và build fail do sai branch.
- Trong /build-webos/oe-core/meta/recipes-support/bmap-tools, thay thế

SRC_URI trong bmap-tools_git.bb:

```
SRC_URI = "git://github.com/intel/${BPN};branch=main;protocol=https"
```

Mục đích:

- SRC_URI cũ có thể trỏ sai repo/branch hoặc link không còn hợp lệ, gây lỗi do_fetch.
- Cập nhật lại để clone đúng repo bmap-tools từ Intel đang được duy trì upstream
- branch=main giúp lấy phiên bản mới nhất, ổn định và tương thích với hệ thống build WebOS/Yocto.
- protocol=https đảm bảo kết nối bảo mật, ít bị chặn bởi firewall và ổn định hơn khi tải từ GitHub.
- Mục đích chung: đảm bảo BitBake fetch source thành công, không làm gián đoạn pipeline build.
- Trong /build-webos/meta-updater/recipes-sota/aktualizr, thay thế dòng cuối của SRC_URI trong aktualizr_git.bb:

```
${@d.expand("https://garage-sign.s3.eu-west-1.amazonaws.com/cli-${GARAGE_SIGN_PV}
↪  .tgz;unpack=0;name=garagesign") if not
↪  oe.types.boolean(d.getVar('GARAGE_SIGN_AUTOVERSION')) else ''} \
```

Mục đích:

- Link tải garage-sign (cli-${GARAGE_SIGN_PV}.tgz) trong recipe có thể đã thay đổi hoặc bị lỗi, làm BitBake do_fetch thất bại.
- Cần thay bằng URL mới hợp lệ để tải đúng phiên bản garage-sign mà WebOS aktualizr yêu cầu.
- unpack=0 và name=garagesign giúp BitBake nhận diện đúng file và xử lý

checksum tương ứng.

- Điều kiện if not GARAGE_SIGN_AUTOVERSION đảm bảo chỉ dùng link thủ công khi autoversion tắt, tránh xung đột version.

2.12 Vá lỗi SMACK

- Trong `/build-webos/meta-webososse/meta-webos-smack/recipes-core/systemd/files`, thay thế nội dung `0001-meson-add-smack-default-process-label-option.patch`:

```
diff --git a/meson.build b/meson.build

index cb9936e23ef44b 100644

a/meson.build

+++ b/meson.build

001085,6 +1085,11 @@ if have

conf.set_quoted('SMACK_RUN_LABEL', get_option('smack-run-label'))

endif

+smack_default_process_label = get_option('smack-default-process-label')

+if smack_default_process_label != ''

+conf.set_quoted('SMACK_DEFAULT_PROCESS_LABEL',

smack_default_process_label)

+endif

+

want_polkit = get_option('polkit')

install_polkit = false

install_polkit_pkla = false

diff --git a/meson_options.txt b/meson_options.txt

index 401f09318b2226 100644

a/meson_options.txt
```

```
+++ b/meson_options.txt

@@ -340,6 +340,8 @@ option('smack', type: 'boolean',

description: 'SMACK support')

option('smack-run-label', type: 'string',

description: 'run systemd-system itself with a specific SMACK label')

+option('smack-default-process-label', type: 'string',

+description: 'run systemd services with a specific SMACK label')

option('polkit', type: 'combo', choices: ['auto', 'true', 'false'],

description: 'polkit support')

option('ima', type: 'boolean',
```

- Patch này bổ sung tùy chọn cấu hình mới cho Meson build system để systemd hỗ trợ thêm tham số smack-default-process-label.
- WebOS OSE sử dụng SMACK để gán nhãn bảo mật cho process/service, nhưng systemd mặc định không có option gán label mặc định cho các service khi chạy.
- Việc thêm option giúp thiết lập nhãn SMACK mặc định ngay trong quá trình build, đảm bảo systemd khởi chạy service với đúng label, tránh lỗi quyền truy cập ở giai đoạn bring-up.
- Mục đích chung: mở rộng khả năng cấu hình SMACK cho systemd, giúp tích hợp bảo mật SMACK vào image WebOS ổn định và đúng cách, không cần sửa thủ công sau build.
- Thay thế nội dung của 0001-fileio.c-fix-build-with-smack-enabled.patch:

```
diff --git a/src/basic/fileio.c b/src/basic/fileio.c index 8d92da3.34902dd 100644
↪  a/src/basic/fileio.c

+++ b/src/basic/fileio.c @@ -24,6 +24,7 @@

#include "parse-util.h" #include "path-util.h" #include "socket-util.h"

+#include "smack-util.h" #include "stdio-util.h" #include "string-util.h" #include
↪  "sync-util.h"

diff --git a/src/basic/meson.build b/src/basic/meson build index 49e1e7ff22caad
↪  100644

a/src/basic/meson.build

+++ b/src/basic/meson.build

@@ -421,7 +421,7 @@ libbasic = static_library('basic',

basic_sources, fundamental_sources, include_directories: basic_includes,

+include_directories: [basic_includes, include_directories('../shared')],
↪  dependencies: [versiondep,

threads,
```

```
libcap,
```

- Patch thêm smack-util.h vì khi bật SMACK, systemd cần thư viện tiện ích để

xử lý nhãn bảo mật, nếu thiếu header sẽ lỗi do_compile.

- Sửa include_directories trong meson.build để bổ sung đường dẫn chứa file SMACK dùng chung, giúp Meson tìm đúng dependency khi build.
- Mục đích chung: fix lỗi biên dịch do thiếu thành phần SMACK, đảm bảo systemd build thành công khi bật bảo mật SMACK trong WebOS OSE.

2.13 Xây dựng Image

- Sau khi hoàn tất các bước chuẩn bị và chỉnh sửa cấu hình, khởi động quá trình xây dựng:

Lệnh	**Ý nghĩa**
source oe-init-build-env	kích hoạt môi trường build Yocto/WebOS trong terminal để bắt đầu chạy BitBake
bitbake webos-image	build và tạo bản image WebOS hoàn chỉnh

- Thời gian build có thể kéo dài lên đến 10 giờ trong lần xây dựng đầu tiên hoặc khi sstate-cache chưa có dữ liệu (trống bộ đệm). Nếu xuất hiện lỗi ở bước do_install hoặc do_compile, nguyên nhân thường là hệ thống không đủ tài nguyên (CPU/RAM/Disk). Khi đó, cần xóa và làm mới bộ đệm build rồi tiến hành build lại để tránh lỗi do thiếu tài nguyên hoặc cache bị lỗi.

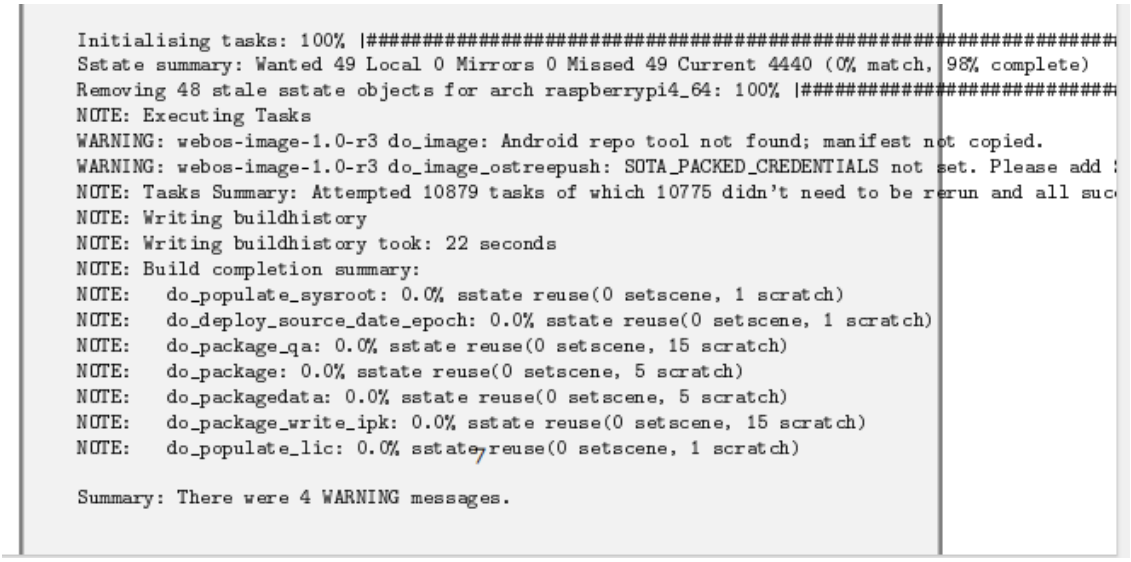
Lệnh	**Ý nghĩa**
bitbake-c clean <recipe-name>	xóa kết quả build của một recipe để build lại từ đầu, tránh lỗi do file cũ còn sót.
bitbake <recipe-name>	build một thành phần cụ thể trong hệ thống Yocto/WebOS

2.14 Kiểm tra kết quả

- Sau khi xây dựng hình ảnh thành công, đầu ra kết quả sẽ như bên dưới:



Hình 5: Kết quả quá trình xây dựng image webOS.



Hình 6: Thông tin đầu ra sau khi build thành công.

- Sau khi xây dựng thành công, hình ảnh hệ điều hành sẽ được lưu tại:

```
/build-webos/BUILD/deploy/images/raspberrypi4-64/webos-image
↪ raspberrypi4-64.rootfs.wic.bz2
```

- Giải nén tệp .wic.bz2 và ghi vào thẻ microSD để sử dụng trên Raspberry Pi 4.
- Sau khi cài đặt, kiểm tra SMACK bằng các lệnh:

```
mount | grep smack
```

Chsmack

smackctl

2.15 Các lỗi trong quá trình Build WebOS

2.16 Lỗi do thiết lập cho việc build quá nhiều nhân, nhiều luồng

- Vấn đề gặp phải

Khi thực thi lệnh build với các tham số -p 0 -b 0, hệ thống sẽ không giới hạn số lượng tiến trình và luồng xử lý, dẫn đến việc tận dụng toàn bộ khả năng của CPU để thực hiện quá trình biên dịch.

Command Line

```
$ sudo scripts/prerequisites.sh
$ ./mcf -p 0 -b 0 raspberrypi4-64
```

Hình 7: Cấu hình mức độ song song của quá trình build.

- Phân tích nguyên nhân
- Việc cấu hình -p 0 -b 0 có thể gây ra các sự cố sau:
- Tình trạng quá tải hệ thống:
- CPU: Tỷ lệ sử dụng đạt mức tối đa 100% → có thể gây ra hiện tượng nhiệt độ tăng cao (trong trường hợp hệ thống làm mát chưa đủ mạnh)

Bộ nhớ RAM: Có nguy cơ bị sử dụng vượt ngưỡng → dẫn đến tình trạng hệ thống bị đơ hoặc kích hoạt cơ chế kernel OOM (Out-of-Memory)

- Vùng Swap: Bị khai thác quá mức → làm cho hệ thống phản hồi cực kỳ chậm chạp hoặc ngừng hoạt động hoàn toàn
- Quá trình build bị lỗi không xác định:
- Một số công cụ/chương trình không hoạt động ổn định khi số lượng tác vụ song song quá lớn
- Hệ thống I/O đĩa cứng bị nghẽn cổ chai → xuất hiện lỗi timeout hoặc không ghi được dữ liệu
- Các thiết bị có cấu hình hạn chế (ví dụ: Raspberry Pi, máy ảo với tài nguyên thấp) không đủ sức chịu tải khi build ở mức tối đa

Giải pháp khắc phục: Điều chỉnh thành -p 6 -b 6 (hoặc -p 3 -b 3)

- Giải thích chi tiết:
- p 6: Thiết lập giới hạn tối đa là 6 tiến trình build có thể hoạt động đồng thời
- -b 6: Mỗi tiến trình được phép sử dụng tối đa 6 luồng xử lý
- Lợi ích của cấu hình này:
- Kiểm soát được lượng công việc thực thi song song → ngăn chặn tình trạng CPU bị khai thác quá mức
- Bảo vệ hệ thống khỏi tình trạng thiếu hụt RAM và hiện tượng đơ máy

- Vẫn tận dụng hiệu quả sức mạnh đa nhân của CPU để tăng tốc quá trình build, nhưng trong phạm vi có thể kiểm soát
- Đảm bảo quá trình biên dịch diễn ra ổn định, hạn chế tối đa nguy cơ xảy ra lỗi do hệ thống quá tải

2.17 Lỗi qt_base do cấu hình meta-qt6 và tràn RAM

- Quá trình build WebOS gặp lỗi liên quan đến Qt6, bao gồm hai vấn đề

chính:

- Lỗi cấu hình meta-qt6: Không thể clone/fetch meta-qt6 layer từ repository
- Lỗi tràn RAM: Khi build qt_base, hệ thống hết bộ nhớ và bị kernel kill process

=> Cả 2 lỗi cần được giải quyết để build thành công

Lỗi qt_base do cấu hình meta-qt6

- URL mặc định của meta-qt6 trong file weboslayers.py không còn hoạt động
- Branch/commit cũ không tồn tại trong repository hiện tại
- WebOS 2.20 yêu cầu Qt 6.4 với commit cụ thể

- Cách chỉnh sửa cấu hình:

- Mở file weboslayers.py
- Tìm và sửa cấu hình meta-qt6
- Tìm dòng chứa meta-qt6 trong file, thay thế bằng:

('meta-qt6', 20, 'https://gitlab.com/qtmirror/yocto-meta-qt6.git',

'branch=6.4,commit=977b3c8e09774400bae7b054d63fc459051f127f',

'meta-qt6'),

- URL mới: https://gitlab.com/qtmirror/yocto-meta-qt6.git (GitLab mirror)
- Branch: 6.4 (tương thích WebOS 2.20)
- Commit: 977b3c8e (verified working)

Lỗi tràn RAM khi build qt_base

- Sau khi sửa xong cấu hình meta-qt6, khi chạy bitbake webos-image, gặp lỗi

tràn RAM:

- Qt rất nặng, cần nhiều bộ nhớ.
- Build quá nhiều tiến trình song song (mặc dù đã dùng -p 6 -b 6)
- Máy có 16GB RAM nhưng vẫn không đủ do:
- Nhiều packages build đồng thời
- qt_base compile cần 4-6GB RAM cho mỗi process
- Hệ điều hành + các service cũng dùng RAM
- Swap không đủ hoặc quá chậm
- Cách chỉnh sửa lỗi tràn RAM: Build lại qt6 riêng

```
bitbake webos-imagebitbake webos-imageSau khi phát hiện lỗi tràn RAM khi build
↪  webos-image, cần build lại Qt packages riêng để tập trung tài nguyên:
```

- Build lại full WebOS image sau khi hoàn thành build qt6 riêng :
- Lợi ích của phương pháp này:
- Tập trung tài nguyên RAM cho Qt packages
- Giảm số lượng processes build đồng thời
- Dễ debug nếu có lỗi
- Giảm nguy cơ tràn RAM
- Qt đã được build sẵn, bitbake sẽ sử dụng lại khi build full image

2.18 Lỗi phiên bản của aktualizr

- Mô tả lỗi

Khi tiến hành build WebOS, quá trình gặp phải sự cố liên quan đến việc tải xuống **GarageSign** - một component phụ thuộc bắt buộc của gói aktualizr. File yêu cầu không thể được download thành công từ địa chỉ đã được cấu hình trong file recipe.

- Nguyên nhân

Package aktualizr yêu cầu tải file nén GarageSign từ một URL trên Internet. Lỗi xảy ra do:

- URL không còn khả dụng:
- Server chính thức đã xóa hoặc di chuyển file
- Địa chỉ trong recipe trỏ đến phiên bảns cũ không còn tồn tại
- Checksum không khớp:
- Khi thử tải từ nguồn thay thế, file tải về có mã MD5/SHA256 khác với giá trị trong recipe
- Hệ thống từ chối file vì nghi ngờ bị thay đổi hoặc lỗi

=> Báo lỗi: "Fetcher failure: Unable to fetch URL from any source"

- Thực hiện các bước chỉnh sửa:
- Đổi phiên bản của công cụ GarageSign
- Cập nhật biến phiên bản từ GARAGE_SIGN_PV = "0.7.4" (hoặc phiên bản cũ khác) sang GARAGE_SIGN_PV = "0.7.7"
- Version 0.7.7 trỏ đến file .tgz mới còn được lưu giữ trên server, trong khi version cũ đã bị loại bỏ
- Cập nhật danh sách địa chỉ nguồn (SRC_URI): Trong file recipe aktualizr (aktualizr.bb), bổ sung hoặc sửa đổi dòng cuối cùng của danh sách nguồn để chỉ định 2 URL mới:
- URL đầu tiên trỏ đến server chính thức cung cấp tool (domain tuf-cli-releases)
- URL thứ hai làm nguồn backup trên Amazon S3 (domain garage-sign)
- Cả hai URL này đều áp dụng version mới được thiết lập (0.7.7).
- Cập nhật mã kiểm tra tính toàn vẹn file
- Mọi file tải về đều cần giá trị kiểm tra (md5sum và sha256sum) để hệ thống xác nhận tính chính xác, tránh lỗi hoặc sửa đổi

- Khi chuyển version, giá trị checksum của file mới đương nhiên khác với file cũ
- Lưu lại giá trị checksum của file version mới:
- Giá trị MD5 mới: 138fc97c7130258efa80865a83290ad1
- Giá trị SHA256 mới: `16d9eef5a3144fbddf74ec206714ce2c526f4b68d8259da7fb5004f284848d59`
- Sau đó, thay thế giá trị cũ bằng giá trị mới trong recipe
- Lưu ý quan trọng khi sửa
- Nếu không thay đổi checksum, hệ thống sẽ xuất hiện lỗi: file đã tải về nhưng kiểm tra không đồng nhất
- Thông báo lỗi sẽ chỉ rõ file .tgz có giá trị MD5 hoặc SHA256 không tương ứng với giá trị mong đợi, bắt buộc phải điều chỉnh checksum
- Vì vậy, cần kiểm tra cẩn thận và xác nhận chính xác giá trị checksum trước khi điều chỉnh

2.19 Quy trình build lại an toàn

1. SSH vào đúng máy chủ: ssh lg_lab_server7@10.10.16.245.
2. Kiểm tra không có build khác đang chạy bằng ps aux | grep -E 'bitbake|cooker'.
3. Kiểm tra RAM, swap, dung lượng và inode.
4. Vào ~/build-webos và nạp source oe-init-build-env.
5. Kiểm tra which bitbake và bitbake –version.
6. Mở tmux rồi chạy bitbake webos-image.
7. Mở cửa sổ SSH thứ hai chạy htop và theo dõi log.
8. Nếu lỗi, đọc dòng lỗi đầu tiên trong log task; không chỉ nhìn dòng FAILED cuối cùng.
9. Chỉ clean recipe bị lỗi; tránh xóa toàn bộ BUILD, downloads hoặc sstate-cache.
10. Sau khi build thành công, ghi lại đường dẫn image và checksum để bàn giao.

3 Phát triển tường lửa

3.1 Yêu cầu

Sau khi cài đặt thành công hệ điều hành WebOS trên nền tảng Raspberry Pi 4, tiến hành triển khai cơ chế tường lửa bằng iptables nhằm tăng cường khả năng bảo mật cho hệ thống.

Trước hết, kiểm tra trạng thái hoạt động của iptables bằng cách sử dụng máy thật hoặc máy ảo để kết nối SSH vào Raspberry Pi thông qua lệnh ssh

```
root@<địa_chỉ_IP_của_Raspberry>
```

Sau đó thực thi các lệnh cần thiết để hiển thị cấu hình và trạng thái hiện tại của iptables. Tiếp theo, tiến hành nạp các tập luật (rules) cho iptables nhằm phát hiện và ngăn

chặn các hành vi dò quét cổng (port scanning), cũng như các hình thức tấn công từ chối dịch vụ (DoS/DDoS).

Sau khi cấu hình hoàn tất, sử dụng máy thật hoặc máy ảo để mô phỏng và kiểm thử khả năng phòng vệ của WebOS trên Raspberry Pi thông qua một số kiểu tấn công mạng phổ biến, bao gồm:

- Tấn công SYN Flood: sử dụng công cụ hping3 để gửi liên tục các gói tin SYN tới cổng 80 của mục tiêu nhằm gây quá tải tài nguyên:

```
hping3 -S -p 80 192.168.1.185 --flood
```

- Tấn công với địa chỉ nguồn ngẫu nhiên (Random Source Attack): tạo lưu lượng SYN flood với địa chỉ IP nguồn giả mạo và ngẫu nhiên:

```
hping3 -S -p 80 192.168.1.185 --flood --rand-source
```

- Tấn công Smurf: phát tán các gói ICMP với địa chỉ nguồn bị giả mạo nhằm khuếch đại lưu lượng tấn công:

```
hping3 --icmp --flood --spoof 192.168.1.255 192.168.1.185
```

- Tấn công LAND: gửi các gói tin SYN với địa chỉ IP nguồn và đích trùng nhau nhằm gây lỗi trong quá trình xử lý kết nối của hệ thống: hping3 -S -p 80 192.168.1.185 -a 192.168.1.185

3.2 Các rule iptables

```
RPI_IP="192.168.1.185"
FIREWALL_CONF="/etc/firewall/whitelist.conf"
echo "=========================================="
echo "BẮT ĐẦU TRIỂN KHAI TƯỜNG LỬA IPTABLES..."
echo "=========================================="
# 1. Làm sạch tường lửa (Tránh xung đột rule cũ)
echo "[1/8] Đang xóa các rule cũ..."
iptables -F
iptables -X
# 2. Thiết lập chính sách mặc định (Default Deny)
echo "[2/8] Thiết lập chính sách Default DROP cho INPUT và FORWARD..."
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
# 3. Kết nối an toàn cơ bản
echo "[3/8] Cho phép Localhost và các kết nối đang hoạt động (ESTABLISHED)..."
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# 4. Mở cổng dịch vụ thiết yếu
echo "[4/8] Mở cổng 22 (SSH)..."
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
# 5. Chống dò quét cổng (Port Scanning) và gói tin lỗi
echo "[5/8] Bật khiên chống Port Scanning và chặn gói tin INVALID..."
iptables -A INPUT -m conntrack --ctstate INVALID -j DROP
iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
iptables -A INPUT -p tcp --tcp-flags ALL ALL -j DROP
iptables -A INPUT -p tcp --tcp-flags SYN,FIN SYN,FIN -j DROP
```

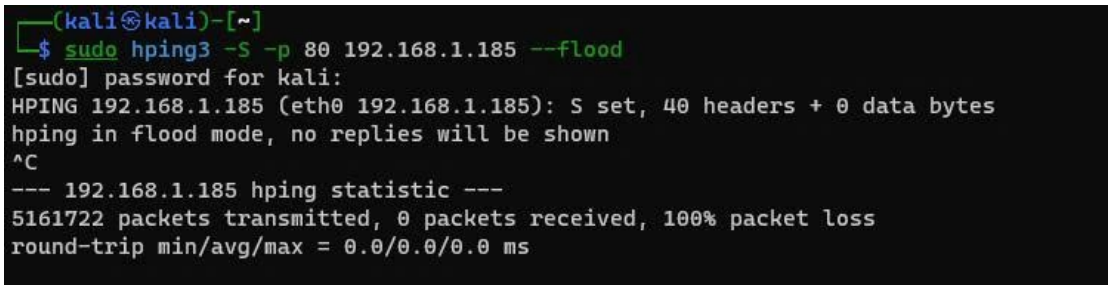
```
# 6. Chống DoS/DDoS (Bảo vệ HTTP - Port 80)
echo "[6/8] Áp dụng Hashlimit chống DoS (Max 20 req/sec)..."
iptables -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -m hashlimit
↪  --hashlimit-name HTTP_DOS --hashlimit-upto 20/sec --hashlimit-burst 20
↪  --hashlimit-mode srcip -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -m conntrack --ctstate NEW -j DROP
# 7. Cho phép Ping (ICMP) để test Smurf Attack
echo "[7/8] Cho phép ICMP (Ping)..."
iptables -A INPUT -p icmp -j ACCEPT
# 8. Chống LAND Attack
echo "[8/8] Chặn IP nội bộ tự gửi gói tin cho chính nó (LAND Attack)..."
iptables -A INPUT -s $RPI_IP -d $RPI_IP -j DROP
iptables -L INPUT -n -v --line-numbers
```

3.3 Thử nghiệm

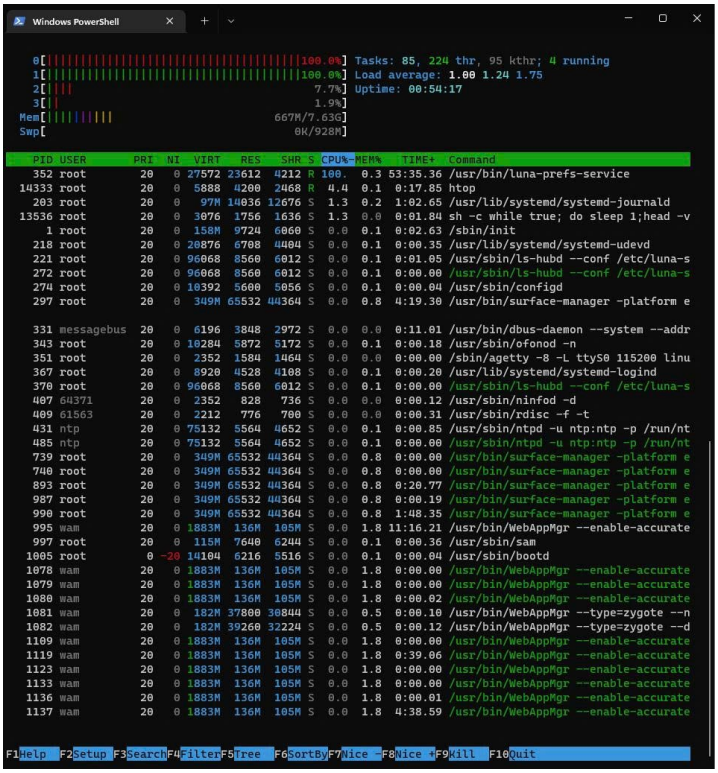
3.3.1 SYN Flood Attack

Khái niệm về SYN flood attack:

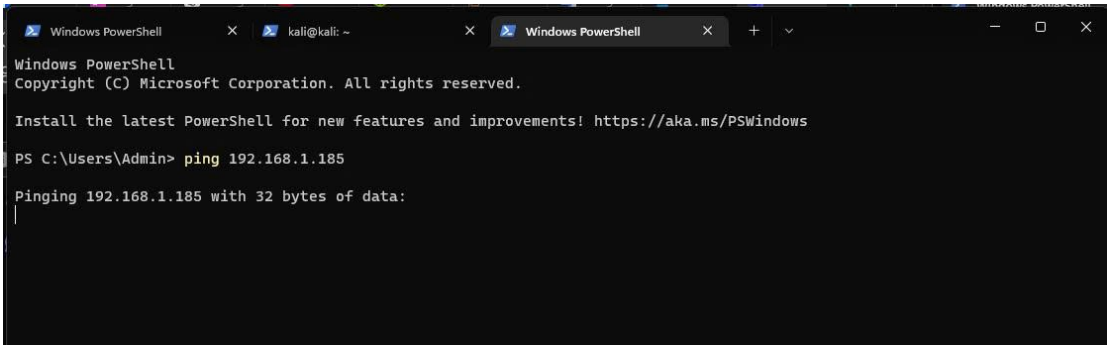
- Tấn công SYN Flood là một dạng tấn công từ chối dịch vụ (Denial of Service - DoS) khai thác cơ chế thiết lập kết nối của giao thức TCP nhằm làm cạn kiệt tài nguyên của máy chủ.
- Trong quá trình thiết lập kết nối TCP thông thường, thiết bị khách (client) sẽ gửi một gói tin có cờ SYN để yêu cầu thiết lập kết nối. Sau khi nhận được gói SYN, máy chủ (server) sẽ phản hồi bằng gói SYN-ACK và chuyển kết nối sang trạng thái SYN_RECV, đồng thời chờ client gửi gói ACK cuối cùng để hoàn tất quá trình bắt tay ba bước (TCP Three-way Handshake).
- Trong cuộc tấn công SYN Flood, kẻ tấn công liên tục gửi một số lượng lớn gói tin SYN đến máy chủ nhưng không hoàn tất bước cuối của quá trình bắt tay, tức là không gửi gói ACK hoặc sử dụng địa chỉ IP nguồn giả mạo khiến phản hồi SYN-ACK không bao giờ được xác nhận.
- Hậu quả là máy chủ phải duy trì hàng loạt kết nối chưa hoàn tất (half-open connections) trong bảng theo dõi kết nối và bộ nhớ đệm (SYN backlog queue). Khi số lượng các kết nối này vượt quá khả năng xử lý của hệ thống, máy chủ sẽ tiêu tốn nhiều tài nguyên như bộ nhớ và CPU, đồng thời không thể tiếp nhận thêm các kết nối hợp lệ từ người dùng, dẫn đến tình trạng suy giảm hiệu năng hoặc từ chối dịch vụ.
- Mục đích của SYN Flood Attack là làm cạn kiệt tài nguyên của máy chủ hoặc dịch vụ TCP, khiến các yêu cầu hợp lệ không được xử lý, từ đó gây gián đoạn hoặc ngừng hoạt động của dịch vụ.



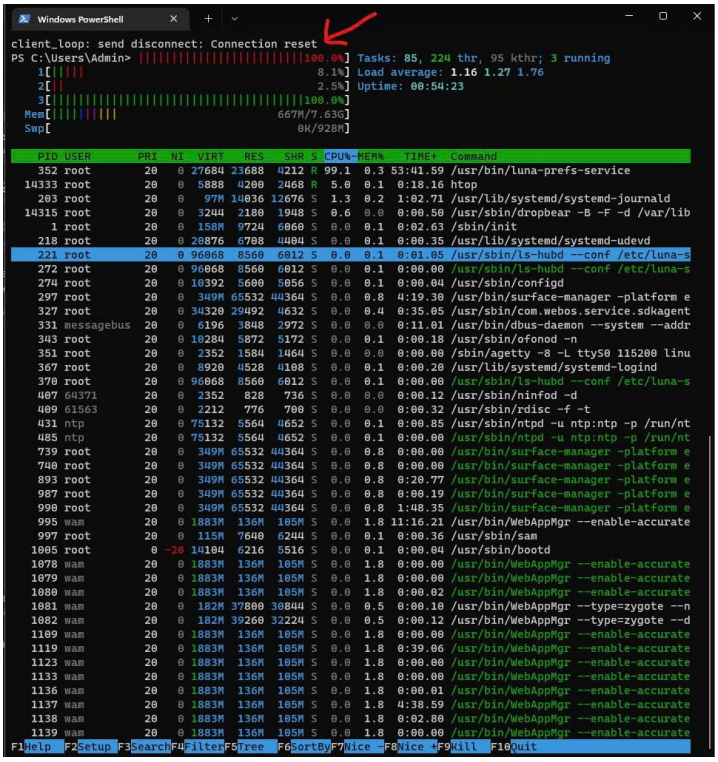
Hình 8: Lệnh tấn công SYN flood đến hệ thống đích.



Hình 9: Theo dõi tài nguyên hệ thống khi thử nghiệm.



Hình 10: Trạng thái kết nối đến hệ thống đích.

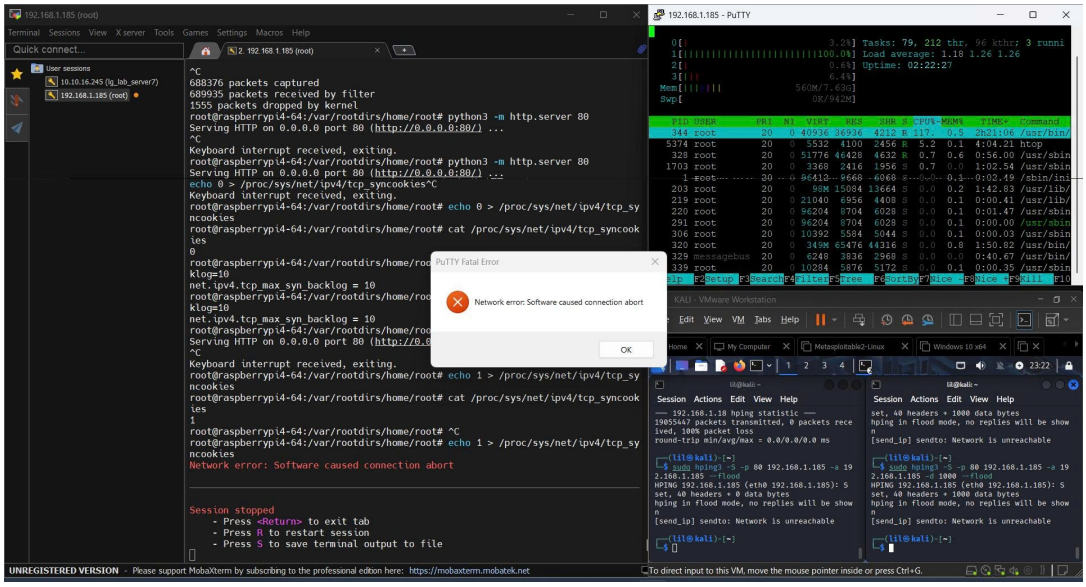


Hình 11: Hệ thống SSH bị disconnect trong thử nghiệm SYN flood.

3.3.2 LAND Attack

Khái niệm Land Attack:

- Tấn công LAND (Local Area Network Denial) là một dạng tấn công từ chối dịch vụ (DoS), trong đó kẻ tấn công gửi các gói TCP SYN có địa chỉ IP nguồn trùng với địa chỉ IP đích, đồng thời cổng nguồn và cổng đích cũng có thể được đặt giống nhau nhằm tạo ra các gói tin có cấu trúc bất thường.
- Khi máy chủ nhận được các gói tin này, hệ điều hành phải kiểm tra và xử lý những kết nối có thông tin nguồn và đích giống nhau. Trên các hệ điều hành hoặc thiết bị mạng cũ, việc xử lý không đúng các gói tin này có thể dẫn đến tiêu tốn tài nguyên, giảm hiệu năng hoặc gây treo hệ thống.
- Mục đích của LAND Attack là khai thác các điểm yếu trong cơ 1chế xử lý gói tin TCP/IP của hệ điều hành nhằm làm suy giảm hiệu năng hoặc gây từ chối dịch vụ. Tuy nhiên, trên các hệ điều hành hiện đại như Linux/WebOS OSE, nhân hệ điều hành đã được bổ sung các cơ chế kiểm tra và loại bỏ các gói tin bất thường, do đó cuộc tấn công thường không còn gây ra hiện tượng treo hoặc sập hệ thống như trên các hệ thống cũ.



Hình 12: Môi trường thực hiện thử nghiệm LAND Attack.

- Để gửi liên tục các gói TCP SYN đến cổng 80 của Raspberry Pi. Mỗi gói tin có 1000 byte payload (-d 1000) và địa chỉ IP nguồn được giả mạo trùng với địa chỉ IP đích (-a 192.168.1.185), mô phỏng cuộc tấn công LAND Attack. Tùy chọn –flood giúp gửi các gói tin với tốc độ tối đa nhằm đánh giá khả năng chống chịu của hệ thống trước lưu lượng bất thường.

3.3.3 Smurf Attack

Khái niệm Smurf Attack:

- Đây là phương thức tấn công từ chối dịch vụ có tính khuếch đại (amplification DoS) được gọi là Smurf Attack.
- Trong kỹ thuật tấn công này, kẻ xấu truyền một lượng lớn các gói tin ICMP Echo Request (hay còn gọi là "ping") đến địa chỉ broadcast của mạng mục tiêu. Yếu tố then chốt ở đây là địa chỉ IP nguồn trong từng gói tin đã bị làm giả thành địa chỉ IP của nạn nhân.
- Khi các gói tin này đến địa chỉ broadcast, toàn bộ các thiết bị trong mạng đều nhận được yêu cầu ping. Mỗi thiết bị sẽ cùng lúc gửi trả lời bằng gói tin ICMP Echo Reply về địa chỉ IP đã bị giả mạo - chính là máy nạn nhân. Hệ quả là nạn nhân bị "tấn công dồn dập" bởi hàng loạt gói phản hồi từ vô số máy tính trong cùng thời điểm.
- Tác động của loại tấn công này là băng thông mạng của nạn nhân bị nghẽn hoàn toàn và thiết bị mục tiêu buộc phải xử lý khối lượng khổng lồ lưu lượng ngoài ý muốn, gây ra tình trạng ngừng trệ hoặc sụp đổ dịch vụ. Mức độ nguy hiểm của Smurf Attack nằm ở đặc tính khuếch đại: chỉ cần một yêu cầu ping đơn lẻ có thể tạo ra vô số phản hồi cùng một lúc.
- Mục đích chủ yếu của phương thức tấn công này là làm tê liệt hoạt động mạng của nạn nhân thông qua việc lợi dụng cơ sở hạ tầng của toàn bộ một mạng khác để "bắn phá" lưu lượng về địa chỉ đã bị giả mạo.

```
┌──(kali㉿kali)-[~]
└─$ sudo hping3 --icmp --flood --spoof 192.168.1.185 192.168.1.255
HPING 192.168.1.255 (eth0 192.168.1.255): icmp mode set, 28 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Hình 13: Lệnh tấn công trong thử nghiệm Smurf Attack.

```
Windows PowerShell  Windows PowerShell  Windows PowerShell
11:00:06.514998 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 49046, length 8
11:00:06.515120 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 49302, length 8
11:00:06.515356 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 49558, length 8
11:00:06.515469 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 49814, length 8
11:00:06.515668 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 50070, length 8
11:00:06.515785 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 50326, length 8
11:00:06.516038 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 50582, length 8
11:00:06.516131 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 50838, length 8
11:00:06.516342 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 51094, length 8
11:00:06.516455 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 51350, length 8
11:00:06.516669 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 51606, length 8
11:00:06.516767 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 51862, length 8
11:00:06.516962 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 52118, length 8
11:00:06.517118 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 52374, length 8
11:00:06.517317 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 52630, length 8
11:00:06.517430 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 52886, length 8
11:00:06.517640 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 53142, length 8
11:00:06.517770 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 53398, length 8
11:00:06.517979 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 53654, length 8
11:00:06.518075 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 53910, length 8
11:00:06.518263 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 54166, length 8
11:00:06.518377 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 54422, length 8
11:00:06.518608 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 54678, length 8
11:00:06.518761 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 54934, length 8
11:00:06.518955 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 55190, length 8
11:00:06.519059 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 55446, length 8
11:00:06.519255 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 55702, length 8
11:00:06.519394 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 55958, length 8
11:00:06.519671 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 56214, length 8
11:00:06.519800 eth0  B   IP raspberrypi4-64 > 192.168.1.255: ICMP echo request, id 15447, seq 56470, length 8
```

Hình 14: Theo dõi gói ICMP tại máy nạn nhân.

Máy nạn nhân nhận được gói tin icmp

Mô hình thử nghiệm

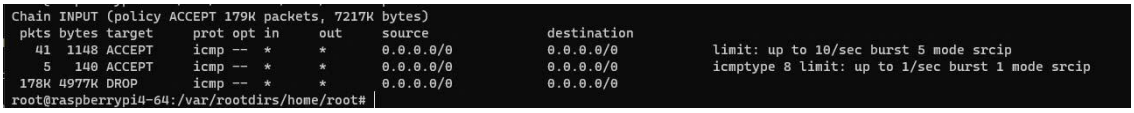
- Máy tấn công: Kali Linux
- Máy nạn nhân: Raspberry Pi
- Broadcast address của mạng LAN: 192.168.1.255

Kết quả bắt gói trên Raspberry Pi:

- IP raspberrypi4-64 > 192.168.1.255: ICMP echo request
- Có thể thấy các gói ICMP Echo Request đã được gửi tới địa chỉ broadcast của mạng.
- Tuy nhiên, hệ thống không ghi nhận số lượng lớn ICMP Echo Reply từ các máy khác trong mạng quay trở lại Raspberry Pi.

Kết luận: Thử nghiệm đã thành công trong việc mô phỏng giai đoạn gửi ICMP Echo Request tới địa chỉ broadcast với địa chỉ nguồn bị giả mạo. Tuy nhiên, cuộc tấn công không tạo được hiệu ứng khuếch đại lưu lượng do các cơ chế bảo vệ chống Smurf đã được tích hợp sẵn trong hệ điều hành và thiết bị mạng hiện đại. Điều này cho thấy Smurf Attack hiện nay ít còn hiệu quả trong các môi trường mạng được cấu hình đúng và sử dụng các hệ điều hành hiện đại

Triển khai Rule chống ICMP Flood và Smurf Attack bằng iptables:



Hình 15: Rule iptables phục vụ kiểm thử ICMP.

Mặc dù các hệ điều hành Linux hiện đại đã mặc định bật cơ chế `icmp_echo_ignore_broadcasts=1` nhằm ngăn máy phản hồi ICMP Echo Request gửi tới địa chỉ broadcast, việc bổ sung các rule trên iptables vẫn cần thiết nhằm tạo thêm một lớp phòng thủ ở tầng host-based firewall.

Việc bổ sung rule iptables mang lại các lợi ích sau:

- Giới hạn tốc độ xử lý ICMP nhằm giảm tải CPU và tài nguyên hệ thống khi xảy ra ICMP Flood hoặc Ping Flood.
- Hạn chế khả năng khai thác từ các host nội bộ chưa được cấu hình đúng.
- Tạo cơ chế phòng thủ theo nguyên tắc Defense in Depth (phòng thủ nhiều lớp).
- Cho phép ghi nhận và thống kê số lượng gói tin bị giới hạn hoặc bị chặn.

Trong quá trình thử nghiệm, nhóm đã triển khai rule giới hạn ICMP Echo Request theo từng địa chỉ IP nguồn bằng module ‘hashlimit‘:

```
iptables -A INPUT -p icmp --icmp-type echo-request \
-m hashlimit \
--hashlimit-upto 1/sec \
--hashlimit-burst 1 \
--hashlimit-mode srcip \
--hashlimit-name ICMP_LIMIT \
-j ACCEPT
```

```
iptables -A INPUT -p icmp -j DROP
```

Rule trên cho phép mỗi địa chỉ IP nguồn gửi tối đa một ICMP Echo Request mỗi giây. Các gói tin vượt quá ngưỡng này sẽ bị loại bỏ bởi rule kế tiếp.

Cơ chế này không trực tiếp ngăn chặn Smurf Attack cổ điển, do Smurf Attack chủ yếu dựa vào việc các thiết bị trong mạng phản hồi ICMP Broadcast. Tuy nhiên, nó giúp giảm tác động của ICMP Flood và hạn chế khả năng một host đơn lẻ tạo ra lượng lớn ICMP traffic tới Raspberry Pi.

Kiểm tra hoạt động của rule:

Để xác minh rule hoạt động đúng, nhóm sử dụng công cụ tạo lưu lượng ICMP với tốc độ cao từ máy Kali trong môi trường lab riêng biệt.

Trên Raspberry Pi, tiến hành theo dõi lưu lượng bằng lệnh:

```
tcpdump -i any icmp
```

Đồng thời kiểm tra bộ đếm packet của iptables:

```
iptables -L INPUT -v -n
```

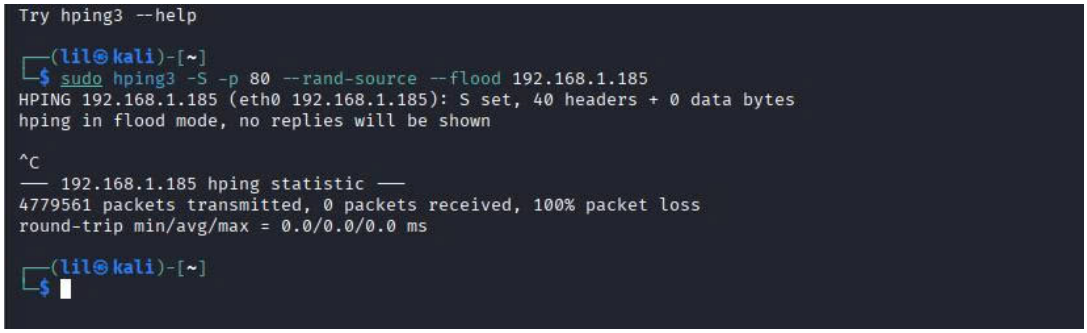
Khi lưu lượng ICMP vượt quá giới hạn đã cấu hình, bộ đếm của rule 'ICMP_LIMIT' chỉ tăng rất chậm hoặc giữ nguyên trong khi bộ đếm của rule 'DROP' tăng lên liên tục. Điều này chứng minh firewall đã thực hiện giới hạn và loại bỏ các ICMP Echo Request vượt ngưỡng cấu hình.

Kết quả thử nghiệm cho thấy cơ chế rate limiting hoạt động đúng như mong đợi và giúp giảm đáng kể số lượng ICMP packet được chuyển tới tầng ứng dụng và nhân hệ điều hành.

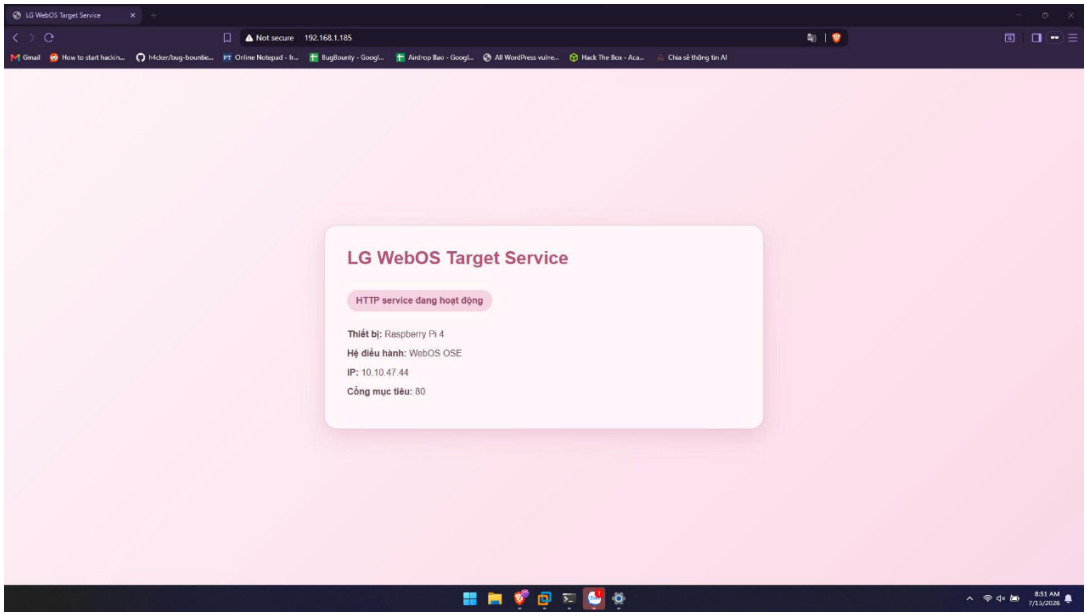
3.3.4 Random Source Attack

Khái niệm Random Source Attack:

- Random Source Attack là một biến thể của SYN Flood Attack, trong đó kẻ tấn công kết hợp việc gửi số lượng lớn gói TCP SYN với kỹ thuật giả mạo địa chỉ IP nguồn (IP Spoofing). Mỗi gói tin được gắn một địa chỉ IP nguồn giả mạo được lựa chọn ngẫu nhiên thay vì sử dụng địa chỉ IP thực của máy tấn công.
- Việc sử dụng các địa chỉ IP nguồn ngẫu nhiên nhằm đạt hai mục tiêu chính:
- Thứ nhất, làm cho máy chủ và firewall khó xác định nguồn tấn công thực sự, từ đó giảm hiệu quả của các cơ chế chặn dựa trên địa chỉ IP.
- Thứ hai, buộc máy chủ phải duy trì đồng thời một số lượng lớn kết nối chưa hoàn tất (half-open connections) đến nhiều địa chỉ IP giả mạo khác nhau, làm tiêu tốn tài nguyên hệ thống.
- Khi số lượng kết nối chưa hoàn tất vượt quá khả năng lưu trữ của hàng đợi SYN hoặc bảng theo dõi kết nối, máy chủ sẽ không thể tiếp nhận thêm các kết nối hợp lệ từ người dùng, dẫn đến hiện tượng suy giảm hiệu năng hoặc từ chối dịch vụ. Đồng thời, máy chủ còn phải gửi các gói SYN-ACK đến các địa chỉ IP giả mạo, làm tăng lưu lượng mạng và tiêu tốn thêm tài nguyên xử lý.
- Random Source Attack khai thác đồng thời cơ chế bắt tay ba bước của giao thức TCP và kỹ thuật giả mạo địa chỉ IP nhằm che giấu nguồn tấn công, gây khó khăn cho việc truy vết và làm giảm hiệu quả của các cơ chế phòng vệ chỉ dựa trên việc giới hạn kết nối theo địa chỉ IP.

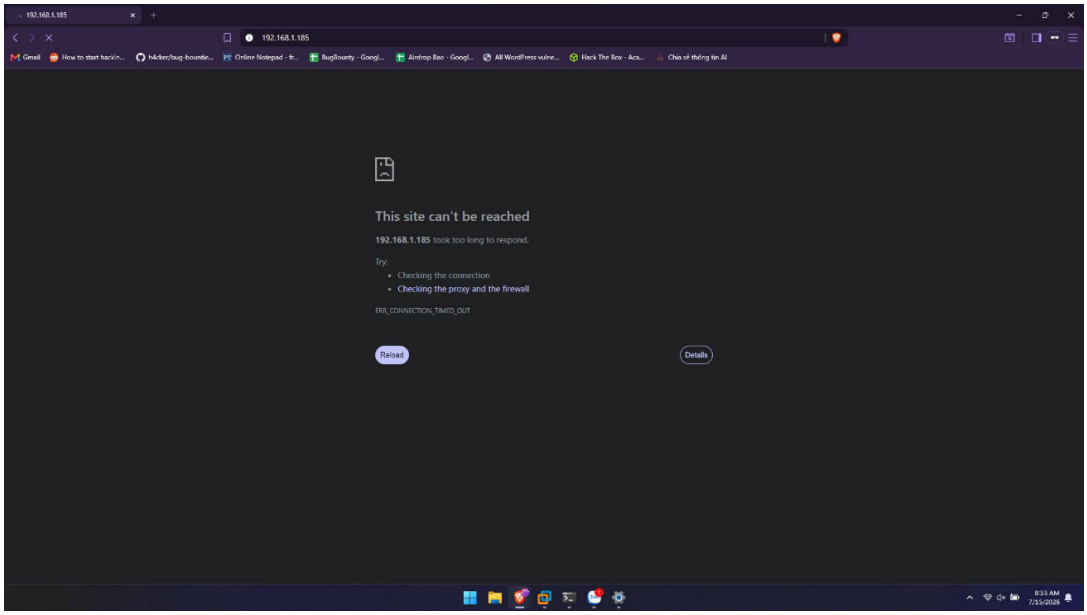


Hình 16: Triển khai lệnh DDoS.



Hình 17: Trạng thái trang web trước khi tấn công.

Trang web đang hoạt động bình thường



Hình 18: Trạng thái trang web sau khi thực hiện tấn công.

Web sập sau khi thi hành lệnh

```
PS C:\Users\Admin> ping 192.168.1.185

Pinging 192.168.1.185 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.1.185:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
PS C:\Users\Admin>
```

Hình 19: Kết quả kiểm tra kết nối sau khi tấn công.

Không ping được, mất luôn kết nối SSH

```
Windows PowerShell   Windows PowerShell
root@raspberrypi4-64:/var/rootdirs/home/root# cd /var/rootdirs/home/root/firewall-monitor/target
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# ls
index.html
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# python3 -m http.server 80 --bind
 0.0.0.0
Traceback (most recent call last):
  File "/usr/lib/python3.10/runpy.py", line 196, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "/usr/lib/python3.10/runpy.py", line 86, in _run_code
    exec(code, run_globals)
  File "/usr/lib/python3.10/http/server.py", line 1307, in <module>
    test(
  File "/usr/lib/python3.10/http/server.py", line 1258, in test
    with ServerClass(addr, HandlerClass) as httpd:
  File "/usr/lib/python3.10/socketserver.py", line 452, in __init__
    self.server_bind()
  File "/usr/lib/python3.10/http/server.py", line 1301, in server_bind
    return super().server_bind()
  File "/usr/lib/python3.10/http/server.py", line 137, in server_bind
    socketserver.TCPServer.server_bind(self)
  File "/usr/lib/python3.10/socketserver.py", line 466, in server_bind
    self.socket.bind(self.server_address)
OSError: [Errno 98] Address already in use
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# netstat -tulnp | grep :80
netstat: invalid option -- 'p'
BusyBox v1.35.0 () multi-call binary.

Usage: netstat [-ral] [-tuwx] [-en]
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# netstat -tuln | grep :80
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# netstat -tuln | grep :80^C
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# fuser 80/tcp
80/tcp:               2022
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# kill -9 2022
root@raspberrypi4-64:/var/rootdirs/home/root/firewall-monitor/target# python3 -m http.server 80 --bind
 0.0.0.0
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.1.114 - - [14/Jul/2026 18:51:21] "GET / HTTP/1.1" 304 -
192.168.1.114 - - [14/Jul/2026 18:51:31] "GET / HTTP/1.1" 200 -
client_loop: send disconnect: Connection reset
```

Hình 20: Thông tin hệ thống trong quá trình kiểm thử.

3.4 Triển khai bash shell iptables

- Kịch bản triển khai bash shell cấu hình iptables

```
### 1: Drop invalid packets ###
```

```
/sbin/iptables -t mangle -A PREROUTING -m conntrack --ctstate INVALID -j DROP
### 2: Drop TCP packets that are new and are not SYN ###
/sbin/iptables -t mangle -A PREROUTING -p tcp ! --syn -m conntrack --ctstate NEW
↪  -j DROP
### 3: Drop SYN packets with suspicious MSS value ###
/sbin/iptables -t mangle -A PREROUTING -p tcp -m conntrack --ctstate NEW -m tcpmss
↪  ! --mss 536:65535 -j DROP
### 4: Block packets with bogus TCP flags ###
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags FIN,SYN,RST,PSH,ACK,URG
↪  NONE -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags FIN,SYN FIN,SYN -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags SYN,RST SYN,RST -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags FIN,RST FIN,RST -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags FIN,ACK FIN -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ACK,URG URG -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ACK,FIN FIN -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ACK,PSH PSH -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ALL ALL -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ALL NONE -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ALL FIN,PSH,URG -j DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ALL SYN,FIN,PSH,URG -j
↪  DROP
/sbin/iptables -t mangle -A PREROUTING -p tcp --tcp-flags ALL SYN,RST,ACK,FIN,URG
↪  -j DROP
### 5: Block spoofed packets ###
/sbin/iptables -t mangle -A PREROUTING -s 224.0.0.0/3 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 169.254.0.0/16 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 172.16.0.0/12 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 192.0.2.0/24 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 192.168.0.0/16 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 10.0.0.0/8 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 0.0.0.0/8 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 240.0.0.0/5 -j DROP
/sbin/iptables -t mangle -A PREROUTING -s 127.0.0.0/8 ! -i lo -j DROP
### 6: Drop ICMP (you usually don't need this protocol) ###
/sbin/iptables -t mangle -A PREROUTING -p icmp -j DROP
### 7: Drop fragments in all chains ###
/sbin/iptables -t mangle -A PREROUTING -f -j DROP
### 8: Limit connections per source IP ###
/sbin/iptables -A INPUT -p tcp -m connlimit --connlimit-above 111 -j REJECT
↪  --reject-with tcp-reset
### 9: Limit RST packets ###
/sbin/iptables -A INPUT -p tcp --tcp-flags RST RST -m limit --limit 2/s
↪  --limit-burst 2 -j ACCEPT
/sbin/iptables -A INPUT -p tcp --tcp-flags RST RST -j DROP
### 10: Limit new TCP connections per second per source IP ###
/sbin/iptables -A INPUT -p tcp -m conntrack --ctstate NEW -m limit --limit 60/s
↪  --limit-burst 20 -j ACCEPT
/sbin/iptables -A INPUT -p tcp -m conntrack --ctstate NEW -j DROP
### 11: Use SYNPROXY on all ports (disables connection limiting rule) ###
# Hidden - unlock content above in "Mitigating SYN Floods With SYNPROXY" section

#Bonus Rules

#Here are some more iptables rules that are useful to increase the overall
↪  security of a Linux server:


### SSH brute-force protection ###
```

```
/sbin/iptables -A INPUT -p tcp --dport ssh -m conntrack --ctstate NEW -m recent
↪  --set
/sbin/iptables -A INPUT -p tcp --dport ssh -m conntrack --ctstate NEW -m recent
↪  --update --seconds 60 --hitcount 10 -j DROP
### Protection against port scanning ###
/sbin/iptables -N port-scanning
/sbin/iptables -A port-scanning -p tcp --tcp-flags SYN,ACK,FIN,RST RST -m limit
↪  --limit 1/s --limit-burst 2 -j RETURN
/sbin/iptables -A port-scanning -j DROP
```

3.5 Các tham số cấu hình iptables

- Loại bỏ các gói tin không hợp lệ:

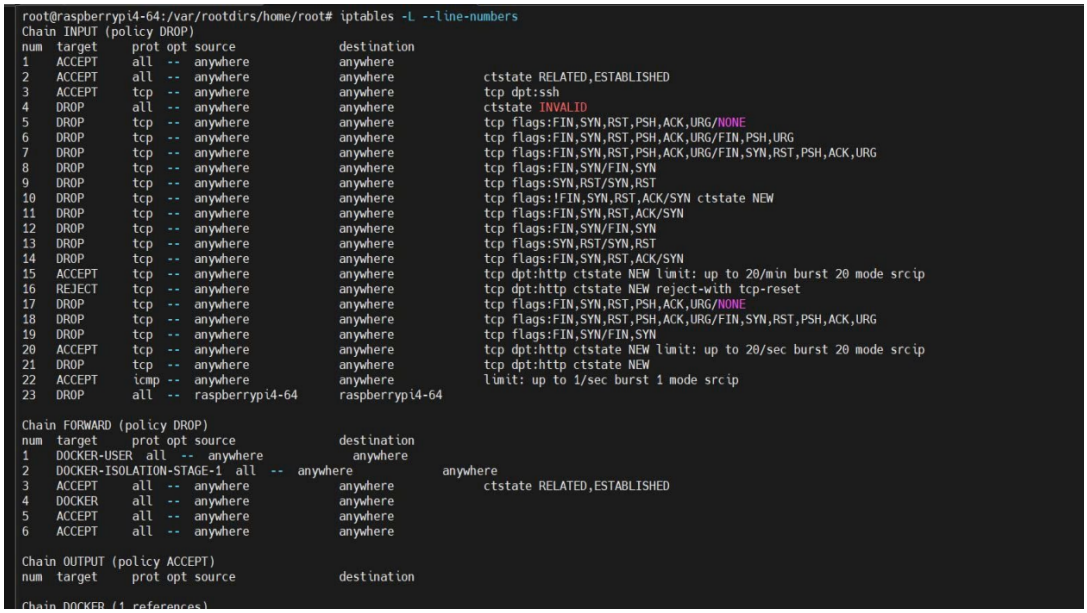
```
iptables -t mangle -A PREROUTING -m conntrack --ctstate INVALID -j DROP
```

- Ý nghĩa tham số:

- -t mangle: Sử dụng bảng mangle để xử lý gói tin trước khi chuyển đến các bảng khác
- -A PREROUTING: Thêm luật vào chuỗi PREROUTING, xử lý ngay khi gói tin vừa đến hệ thống.
- -m conntrack: Sử dụng module theo dõi trạng thái kết nối
- –ctstate INVALID: Chỉ áp dụng đối với các gói tin có trạng thái không hợp lệ
- -j DROP: Hủy bỏ gói tin mà không gửi phản hồi

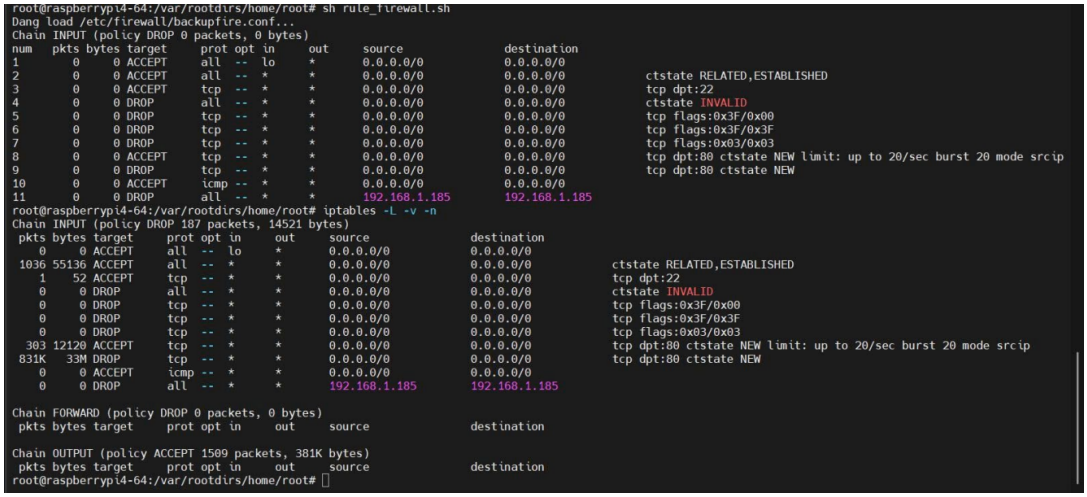
- Vai trò: Trong quá trình tấn công, đặc biệt là TCP Flood hoặc ACK Flood, nhiều gói tin không thuộc bất kỳ kết nối hợp lệ nào sẽ được gửi đến máy chủ. Rule này loại bỏ các gói tin bất thường ngay từ đầu, giảm tải cho kernel và hạn chế việc tiêu tốn CPU cũng như bộ nhớ.

Các rules đã set



Hình 21: Các rule iptables đã được thiết lập.

Rules trước khi thực hiện DDoS và sau khi thực hiện DDoS :



Hình 22: Bộ đếm rule trước và sau khi thực hiện DDoS.

3.6 So sánh khả năng hỗ trợ iptables

So sánh mức độ hỗ trợ iptables của WebOS nguyên bản và WebOS Lab.

- Trong quá trình thiết lập firewall trên hệ điều hành WebOS OSE, nhóm ghi nhận sự khác biệt giữa hai môi trường triển khai:

- WebOS OSE nguyên bản (mặc định)

- WebOS được build tại Lab. Cụ thể, phiên bản WebOS Lab hỗ trợ đầy đủ các chức năng iptables nâng cao, trong khi WebOS OSE nguyên bản bị hạn chế ở một số tính năng.

- Nguyên nhân: Sự khác biệt này xuất phát từ cấu hình kernel trong quá trình build WebOS. Đối với WebOS Lab, file bridge.cfg đã được chỉnh sửa để kích hoạt các module Netfilter mở rộng, bao gồm:

```
CONFIG_NETFILTER_XT_MATCH_CONNBYTES=y
CONFIG_NETFILTER_XT_MATCH_CONNLIMIT=y
CONFIG_NETFILTER_XT_MATCH_IPRANGE=y
CONFIG_NETFILTER_XT_MATCH_LENGTH=y
CONFIG_NETFILTER_XT_MATCH_LIMIT=y
CONFIG_NETFILTER_XT_MATCH_MAC=y
CONFIG_NETFILTER_XT_MATCH_QUOTA=y
CONFIG_NETFILTER_XT_TARGET_NFLOG=y
CONFIG_NF_LOG_IPV4=y
CONFIG_NETFILTER_XT_MATCH_RECENT=y
CONFIG_NETFILTER_XT_MATCH_TCPMSS=y
CONFIG_NETFILTER_XT_TARGET_TCPMSS=y
CONFIG_NETFILTER_XT_MATCH_HASHLIMIT=y
CONFIG_NETFILTER_XT_MATCH_NFACCT=y
```

Iptables không phải là một chương trình độc lập tích hợp sẵn toàn bộ tính năng. Thay vào đó, iptables hoạt động dựa trên các module của kernel Linux, còn gọi là Netfilter modules (xtables extensions). Để một tính năng iptables cụ thể (ví dụ như connbytes hoặc limit) có thể hoạt động, kernel bắt buộc phải được biên dịch với module tương ứng được kích hoạt.

WebOS OSE nguyên bản: Trong WebOS OSE nguyên bản, nhiều khả năng cấu hình kernel mặc định không kích hoạt các tùy chọn:

- CONFIG_NETFILTER_XT_MATCH_CONNBYTES
- CONFIG_NETFILTER_XT_MATCH_LIMIT
- Do đó, các module này không được biên dịch vào kernel, khiến iptables không thể tìm thấy hoặc sử dụng các tính năng tương ứng.

WebOS OSE build tại Lab:

- Ngược lại, trong phiên bản WebOS OSE được build tại Lab, nhóm đã chủ động kích hoạt các tùy chọn:
- CONFIG_NETFILTER_XT_MATCH_CONNBYTES=y
- CONFIG_NETFILTER_XT_MATCH_LIMIT=y

trong quá trình cấu hình và biên dịch kernel tùy chỉnh. Việc này đảm bảo các module Netfilter cần thiết được tích hợp trực tiếp vào kernel, cho phép iptables sử dụng đầy đủ các tính năng nâng cao.

```
WEBOS_IMAGE_EXTRA_INSTALL:append = " \
    htop \
    tcpdump \
    kernel-module-xt-hashlimit \
    ipset \
    apt \
"
```

Yêu cầu hệ thống xây dựng WebOS thêm các gói phần mềm được liệt kê vào ảnh hệ thống cuối cùng trong đó có kernel-module-xt-hashlimit là gói chứa module kernel cho iptables extension hashlimit